

基于运动学与视线遮挡判据的烟幕干扰弹投放策略研究

摘要

针对无人机投放烟幕干扰弹以遮蔽来袭导弹对真目标探测的问题,本文建立了导弹匀速直指假目标、无人机等高度匀速直线飞行、烟幕弹自由落体与云团匀速下沉的运动学模型,并以“导弹—真目标几何中心视线被半径 10 m 烟幕球遮挡”作为有效遮蔽判据;多弹情形取遮蔽时间集合的并测度。对问题 1,给定策略下 FY1 对 M1 的有效遮蔽时长为 1.43 s。对问题 2,经航向邻域直接搜索,单弹遮蔽时长提升至 4.71 s。对问题 3, FY1 同航迹投放 3 枚弹,并集遮蔽时长达 5.16 s。对问题 4, FY1/FY2/FY3 各投 1 枚并互补时序,并集遮蔽时长达 15.82 s。对问题 5,采用“任务分配—单机多弹优化”分层策略,五机协同使 M1/M2/M3 遮蔽时长分别达 10.00/4.76/5.24 s,合计 20.00 s。结果表明:视线落点匹配与多平台时序互补是延长有效遮蔽的关键。

关键字: 烟幕干扰 视线遮挡 运动学模型 分层优化 多机协同

Abstract

To screen incoming missiles from detecting the true target, we model the kinematics of missiles, UAVs, free-fall bombs and sinking smoke clouds, and define effective screening by a 10 m line-of-sight (LOS) occlusion criterion; multi-bomb coverage is measured by the union of screening intervals. Problem 1 yields 1.43 s under the given strategy. Problem 2 improves single-bomb screening to 4.71 s. Problem 3 deploys three bombs from FY1 and reaches 5.16 s by union. Problem 4 coordinates FY1/FY2/FY3 (one bomb each) and achieves 15.82 s. Problem 5 uses a hierarchical assign-then-optimize scheme for five UAVs against M1/M2/M3, obtaining 10.00/4.76/5.24 s respectively and 20.00 s in total. LOS placement and staggered multi-platform timing are shown to be essential.

Keywords: smoke screening; line-of-sight occlusion; kinematics model; hierarchical optimization; multi-UAV coordination

插图清单

1	问题 1: 烟幕球心到导弹—真目标视线距离随时间变化	15
2	问题 1 三维空间态势示意	15
3	问题 2: 烟幕球心到导弹—真目标视线距离随时间变化	20
4	问题 2 三维空间态势示意	20
5	问题 1 与问题 2 策略对比	21
6	问题 3: 三弹协同下球心到视线最小距离随时间变化	25
7	问题 3: 各弹距离曲线与并集有效区间	25
8	问题 3: 三弹有效遮蔽时序图	25
9	问题 3: 三弹起爆点水平俯视图	26
10	问题 1-3 有效遮蔽时长对比	26
11	问题 4: 三机协同遮蔽距离曲线	29
12	问题 4: 各机距离曲线与并集有效区	30
13	问题 4: 三机起点与起爆点俯视图	30
14	问题 4: 单机贡献与并集时长对比	30
15	问题 4: 三机有效遮蔽时序图	31
16	问题 5: M1/M2/M3 遮蔽距离曲线总览	34
17	问题 5: M1 遮蔽距离细部曲线	34
18	问题 5: M2 遮蔽距离细部曲线	35
19	问题 5: M3 遮蔽距离细部曲线	35
20	问题 5: 任务分配与各导弹遮蔽时长	35
21	问题 5: 五机起点、起爆点与三导弹水平态势	36
22	问题 1-5 有效遮蔽时长对比	36

附表清单

1	主要符号说明	10
2	问题 1 部分时刻的中间计算结果	16
3	问题 1 计算结果	16
4	问题 2 优化结果	19
5	问题 1 与问题 2 结果对比	21
6	问题 3 三弹时序与点位明细	24
7	问题 3 优化结果汇总	24
8	问题 4 三机策略明细	28
9	问题 4 结果汇总	29

10	问题 5 主要投放策略摘要	33
11	问题 5: 各导弹遮蔽时长与分配	33

一、问题重述

1.1 问题背景

烟幕干扰弹通过燃烧或爆炸形成烟幕/气溶胶云团，在来袭武器与保护目标之间构成遮蔽，具有成本低、效费比高等优点。本文考虑由长续航无人机挂载烟幕干扰弹，在警戒雷达发现目标后受领任务，于特定空域投放并起爆，以尽量避免来袭空地导弹发现真目标。

1.2 已知条件

以假目标为坐标原点，水平面为 xy 平面。真目标为底面圆心 $(0, 200, 0)$ 、半径 7 m 、高 10 m 的圆柱。导弹飞行速度 300 m/s ，飞行方向直指假目标。烟幕干扰弹起爆后瞬时形成球状云团，并以 3 m/s 匀速下沉；云团中心 10 m 范围内的烟幕在起爆后 20 s 内可提供有效遮蔽。无人机等高度匀速直线飞行，速度范围为 $70 \sim 140\text{ m/s}$ ，航向与速度一经确定不再调整。烟幕弹脱离无人机后仅在重力作用下运动。

警戒雷达发现目标时，三枚导弹与五架无人机初始位置分别为： $M1(20000, 0, 2000)$ ， $M2(19000, 600, 2100)$ ， $M3(18000, -600, 1900)$ ； $FY1(17800, 0, 1800)$ ， $FY2(12000, 1400, 1400)$ ， $FY3(6000, -3000, 700)$ ， $FY4(11000, 2000, 1800)$ ， $FY5(13000, -2000, 1300)$ 。同机连续投放两枚烟幕弹的时间间隔不少于 1 s 。

1.3 求解任务

1. **问题 1:** $FY1$ 以 120 m/s 朝向假目标飞行，受领任务 1.5 s 后投放 1 枚烟幕干扰弹，间隔 3.6 s 后起爆，求对 $M1$ 的有效遮蔽时长。
2. **问题 2:** 确定 $FY1$ 的飞行方向、飞行速度、投放点与起爆点，使对 $M1$ 的有效遮蔽时间尽可能长。
3. **问题 3:** $FY1$ 投放 3 枚烟幕干扰弹干扰 $M1$ ，给出策略并保存至 `result1.xlsx`。
4. **问题 4:** $FY1$ 、 $FY2$ 、 $FY3$ 各投放 1 枚干扰 $M1$ ，给出策略并保存至 `result2.xlsx`。
5. **问题 5:** 5 架无人机（每架至多 3 枚）干扰 $M1$ 、 $M2$ 、 $M3$ ，给出策略并保存至 `result3.xlsx`。

二、问题分析

2.1 总体思路

本问题的核心不是单纯“把烟幕投到导弹飞行路径上”，而是使烟幕在尽可能长的时间区间内阻断导弹对真目标的探测视线。由于导弹始终飞向假目标 O ，而真目标偏置于 $(0, 200)$ 附近，导弹轨迹与“导弹—真目标”视线是两条不同的空间曲线族：前者是

固定射线，后者随导弹位置 $M(t)$ 不断变化。因此，建模必须同时刻画三类运动（导弹、无人机/弹体、云团）以及一条时变几何关系（视线遮挡）。

据此，本文将求解过程分解为：

1. **运动学建模**：给出 $M(t)$ 、 $U(t)$ 、 $B(t)$ 、 $C(t)$ 的解析表达式；
2. **遮蔽判定建模**：将“有效遮蔽”转化为点到线段距离不等式；
3. **指标量化**：把有效时间集合的测度作为目标函数 J ；
4. **分问题求解**：问题 1 做正向仿真；问题 2-4 做单目标约束优化；问题 5 做任务分配与分层优化。

2.2 几何关系与遮蔽机理

记真目标几何中心为 $T = (0, 200, 5)$ 。任意时刻导弹位置为 $M(t)$ ，则探测视线可抽象为线段 MT 。烟幕云团在起爆后为以 $C(t)$ 为心、 $R = 10\text{ m}$ 为半径的球体。若球体与线段 MT 相交（等价于点 $C(t)$ 到线段 MT 的最短距离 $d(t) \leq 10$ ），则认为该时刻视线被遮挡，从而对真目标形成有效遮蔽。

该判据有两方面含义：

1. **空间条件**：云团必须位于导弹与真目标之间或其邻域，而不能仅“靠近导弹轨迹却远离视线”；
2. **时间条件**：即使某一瞬间 $d(t)$ 很小，若不能在连续（或较长）时段内保持，则总遮蔽时长仍然有限。

由于云团起爆后水平位置固定、仅竖直下沉，其水平投影是一个“一次性落点”。这意味着问题 2 的本质是：选择合适的落点（由航向、速度、投放与引信共同决定）以及起爆时刻，使导弹在高速逼近过程中，“扫过”的视线族能够较长时间掠过该固定球心邻域。

2.3 问题 1 的分析

问题 1 给出了全部控制量：航向朝向假目标、速度 120 m/s 、投放时刻 1.5 s 、引信延时 3.6 s 。因此不存在优化自由度，属于**正向仿真问题**。求解路径为：

1. 计算投放点、起爆点与起爆后云团轨迹 $C(t)$ ；
2. 同步推进导弹轨迹 $M(t)$ ；
3. 在 $[t_e, \min(t_e + 20, t_{\text{hit}})]$ 上逐时刻检验 $d(t) \leq 10$ ；
4. 对满足条件的时刻累加得到 J_1 。

该结果既是题目要求的答案，也是后续优化的对照基线：任何改进策略都应至少显著优于 J_1 。

2.4 问题 2 的分析

问题 2 需要同时决定“飞哪里、飞多快、何时投、何时爆”。若直接以三维落点坐标为决策变量，还需反演无人机能否到达，约束耦合较强。更自然的参数化是：

$$\mathbf{x} = (\theta, v, t_d, \tau), \quad (1)$$

其中 θ 控制航向， v 控制水平推进快慢， (t_d, τ) 控制起爆时空位置。四者一经给定，投放点与起爆点便唯一确定，因而“投放点、起爆点”并非独立变量，而是决策变量的导出量。

目标函数 $J(\mathbf{x})$ 具有如下特征：

1. **非光滑性**：由 0-1 判定诱导，导数几乎处处不存在；
2. **多峰性**：不同起爆高度与航向组合可能对应多个局部较好窗口；
3. **计算可替代性**：单次评估只需一次时间离散扫描，成本可控。

因此梯度类方法不适用，宜采用网格搜索、模式搜索或进化算法等直接法。考虑到无人机初始位置接近导弹来向、且“朝向假目标”是合理先验，本文采用“航向邻域粗搜 + 局部加密”的两阶段策略，在精度与计算量之间取得平衡。

2.5 关键时间尺度

对 M1， $\|P_{M0}\| \approx 20100 \text{ m}$ ， $t_{\text{hit}} \approx 67 \text{ s}$ 。云团有效寿命仅 20 s ，无人机速度远低于导弹。这说明：

- 有效遮蔽通常发生在导弹尚未飞临真目标之前的中前期窗口；
- 过晚起爆会导致云团尚未进入视线邻域，导弹已掠过；
- 过早起爆虽可能捕捉早期视线，但若落点偏离，窗口仍然很短。

问题 1 给出的 $t_e = 5.1 \text{ s}$ 属于偏早起爆；问题 2 的优化结果进一步提前到约 2.6 s ，并微调航向，从而拉长有效窗口。

2.6 问题 3 的分析

问题 3 在单机约束下将弹数增至 3 枚：航向与速度仍全局唯一，但增加两组投放/引信参数，并满足相邻投放间隔 $\geq 1 \text{ s}$ 。目标函数改为三枚弹遮蔽时间集合的并

$$J_3 = \text{meas} \left(\bigcup_{i=1}^3 \{t : d_i(t) \leq 10\} \right).$$

由于三弹落在同一直线航迹上，空间多样性有限，增益主要来自起爆时刻沿视线扫掠方向的错峰覆盖。

2.7 问题 4 的分析

问题 4 引入 FY2、FY3，三机可独立选择航向与速度，等价于在不同空域布置三个“一次性落点”。更优策略往往是让三机分别覆盖导弹逼近过程的前、中、后段窗口，使并集长度接近各机单弹时长之和。求解上采用“单机视线落点优化 + 联合时序微调”。

2.8 问题 5 的分析

问题 5 同时面对 3 枚导弹与至多 15 枚弹的组合爆炸，直接全局搜索维数过高。本文采用分层策略：

1. 按相对几何位置将无人机指派给导弹（保证每枚导弹至少一架）；
2. 对每架无人机在其任务导弹上做多弹优化；
3. 以各导弹遮蔽时长之和为总指标。

三、模型假设

为突出主要矛盾并保证模型可解，作如下假设：

1. 忽略风场与空气阻力；烟幕弹水平速度保持投放瞬间无人机速度，竖直方向做初速为 0 的自由落体。
2. 重力加速度取 $g = 9.8 \text{ m/s}^2$ ，地球曲率与科氏力可忽略。
3. 云团视为半径 $R = 10 \text{ m}$ 的球体，球心以 3 m/s 匀速下沉，形状不随时间扩散变形。
4. 有效遮蔽采用中心视线判据：导弹位置与真目标几何中心连线段到烟幕球心的最短距离不大于 10 m ，且处于起爆后 20 s 内。
5. 无人机受领任务后瞬时完成航向与速度调整，之后保持等高度匀速直线飞行，中途不转弯、不变速。
6. 假目标固定于原点，导弹速度大小恒定、方向始终指向假目标，不考虑末段机动。
7. 雷达发现、任务下发与无人机响应之间的延迟已体现在题目给定时序中，不再另设通信延迟变量。

四、符号说明

五、模型建立

5.1 建模框架

本文模型由“对象运动层—遮蔽判定层—优化决策层”三层构成：

1. 对象运动层：分别建立导弹、无人机、自由下落弹体与下沉云团的运动方程组；

表 1 主要符号说明

符号	意义
O	假目标位置（坐标原点）
T	真目标几何中心 $(0, 200, 5)$
P_{M0}	导弹初始位置
U_0	无人机初始位置
v_m	导弹速度，300 m/s
v	无人机水平飞行速度
$\mathbf{h}$	无人机水平航向单位向量
θ	水平航向角（相对 $+x$ 轴）
t_d	投放时刻
τ	引信延时（投放至起爆时间间隔）
t_e	起爆时刻， $t_e = t_d + \tau$
t_{hit}	导弹命中假目标时刻
$M(t)$	t 时刻导弹位置
$U(t)$	t 时刻无人机位置
$B(t)$	t 时刻烟幕弹位置
$C(t)$	t 时刻烟幕球心位置
$d(t)$ 、 $d_i(t)$	球心到线段 $M(t)T$ 的最短距离
$I(t)$ 、 $I_i(t)$	有效遮蔽示性函数
R	有效遮蔽半径，10 m
J 、 J_k	有效遮蔽时长（及问题 k 的目标值）
S_i	第 i 枚弹的有效遮蔽时刻集合
a_{uk}	问题 5 中无人机 u 对导弹 k 的指派变量

2. 遮蔽判定层：将“看见/看不见”转化为几何不等式，并用示性函数定义目标泛函 J ；
3. 优化决策层：在可行域 $\mathcal{X}$ 上求解 $\max_{\mathbf{x} \in \mathcal{X}} J(\mathbf{x})$ （问题 1 退化为给定 $\mathbf{x}$ 的正向计算）。

5.2 导弹运动方程组

导弹以恒定速率 $v_m = 300 \text{ m/s}$ 直指假目标 O 。记初始位置 $P_{M0} = (x_{M0}, y_{M0}, z_{M0})$ ，则

$$\begin{aligned} \mathbf{e}_m &= \frac{-P_{M0}}{\|P_{M0}\|} = \left(-\frac{x_{M0}}{L}, -\frac{y_{M0}}{L}, -\frac{z_{M0}}{L} \right), \quad L = \|P_{M0}\|, \\ M(t) &= (x_M(t), y_M(t), z_M(t)) = P_{M0} + v_m t \mathbf{e}_m, \\ x_M(t) &= x_{M0} - \frac{v_m x_{M0}}{L} t, \quad y_M(t) = y_{M0} - \frac{v_m y_{M0}}{L} t, \quad z_M(t) = z_{M0} - \frac{v_m z_{M0}}{L} t, \\ t_{\text{hit}} &= \frac{L}{v_m}. \end{aligned} \tag{2}$$

对 M1: $L = \sqrt{20000^2 + 2000^2} \approx 20099.75$, $t_{\text{hit}} \approx 67.00 \text{ s}$ 。

5.3 无人机运动方程组

无人机等高度匀速直线飞行，航向角 θ ，速度 v ，初值 $U_0 = (x_{U0}, y_{U0}, z_{U0})$ ：

$$\begin{aligned} \mathbf{h} &= (\cos \theta, \sin \theta, 0), \\ U(t) &= (x_U(t), y_U(t), z_U(t)) = U_0 + vt \mathbf{h}, \\ x_U(t) &= x_{U0} + vt \cos \theta, \quad y_U(t) = y_{U0} + vt \sin \theta, \quad z_U(t) = z_{U0}, \\ 70 &\leq v \leq 140. \end{aligned} \tag{3}$$

5.4 烟幕弹自由下落方程组

设投放时刻 t_d ，引信延时 τ ，起爆时刻 $t_e = t_d + \tau$ 。弹体继承水平速度，竖直初速为 0：

$$\begin{aligned} B(t) &= (x_B(t), y_B(t), z_B(t)), \quad t \in [t_d, t_e], \\ x_B(t) &= x_U(t_d) + v(t - t_d) \cos \theta, \\ y_B(t) &= y_U(t_d) + v(t - t_d) \sin \theta, \\ z_B(t) &= z_{U0} - \frac{1}{2}g(t - t_d)^2, \\ B(t_e) &= \left(x_U(t_d) + v\tau \cos \theta, y_U(t_d) + v\tau \sin \theta, z_{U0} - \frac{1}{2}g\tau^2 \right). \end{aligned} \tag{4}$$

5.5 云团下沉方程组

起爆后云团水平位置固定、竖直以 3 m/s 匀速下沉，有效寿命 20 s：

$$\begin{aligned} C(t) &= (x_C(t), y_C(t), z_C(t)), \quad t \in [t_e, t_e + 20], \\ x_C(t) &= x_B(t_e), \quad y_C(t) = y_B(t_e), \\ z_C(t) &= z_B(t_e) - 3(t - t_e). \end{aligned} \quad (5)$$

5.6 遮蔽判定方程组

真目标几何中心 $T = (0, 200, 5)$ 。点 $C(t)$ 到线段 $M(t)T$ 的最短距离

$$\begin{aligned} \mathbf{w}(t) &= T - M(t), \\ s^*(t) &= \text{clip}\left(\frac{(C(t) - M(t)) \cdot \mathbf{w}(t)}{\mathbf{w}(t) \cdot \mathbf{w}(t)}, 0, 1\right), \\ d(t) &= \|C(t) - M(t) - s^*(t) \mathbf{w}(t)\|. \end{aligned} \quad (6)$$

定义示性函数

$$I(t) = \begin{cases} 1, & d(t) \leq R = 10 \text{ 且 } t_e \leq t \leq \min(t_e + 20, t_{\text{hit}}), \\ 0, & \text{否则.} \end{cases} \quad (7)$$

单弹有效遮蔽时长（目标泛函）为

$$J = \int_0^{t_{\text{hit}}} I(t) dt = \text{meas}\{t : I(t) = 1\} \approx \Delta t \sum_k I(t_k). \quad (8)$$

5.7 多弹并集目标泛函

设第 i 枚弹对应 $d_i(t)$ 、 $I_i(t)$ ，则并集目标

$$\begin{aligned} I_{\cup}(t) &= \max_i I_i(t) = \mathbf{1}_{\{\min_i d_i(t) \leq 10\}}, \\ J_{\text{union}} &= \int_0^{t_{\text{hit}}} I_{\cup}(t) dt \approx \Delta t \sum_k I_{\cup}(t_k). \end{aligned} \quad (9)$$

各问的完整数学规划模型见相应求解章节；公共可行域记为

$$\mathcal{F}_0 = \{(\theta, v, t_d, \tau) : 70 \leq v \leq 140, t_d \geq 0, \tau > 0, z_B(t_e) > 0, t_e < t_{\text{hit}}\}. \quad (10)$$

六、问题 1 的求解

6.1 数学模型

问题 1 为给定策略下的正向评估，决策变量全部固定，需求解目标值 J_1 。

已知参数（等式约束）

$$\begin{aligned} U_0 &= (17800, 0, 1800), \quad P_{M0} = (20000, 0, 2000), \\ v &= 120, \quad \theta = \pi \left(\mathbf{h} = (-1, 0, 0) \right), \\ t_d &= 1.5, \quad \tau = 3.6, \quad t_e = t_d + \tau = 5.1. \end{aligned} \quad (11)$$

状态方程组（由式(2)–(5)特化）

$$\begin{aligned} M(t) &= (20000, 0, 2000) + 300t \mathbf{e}_m, \\ U(t) &= (17800 - 120t, 0, 1800), \\ B(t) &= U(1.5) + (-120, 0, 0)(t - 1.5) - \frac{1}{2}g(t - 1.5)^2 \mathbf{e}_z, \\ C(t) &= (B_x(5.1), 0, B_z(5.1) - 3(t - 5.1)), \quad t \in [5.1, 25.1]. \end{aligned} \quad (12)$$

目标函数

$$J_1 = \int_{5.1}^{25.1} \mathbf{1}_{\{d(t) \leq 10\}} dt, \quad (13)$$

其中 $d(t)$ 由式(6)计算。本问无自由决策变量，属于确定性仿真问题。

6.2 求解目标与已知量

问题 1 要求在全部控制量给定的条件下求 J_1 。求解流程为“先求关键点位，再做时间扫描，最后统计有效时长”。

6.3 步骤一：确定无人机航迹与投放点

由等高度匀速直线飞行模型，

$$U(t) = U_0 + vt\mathbf{h} = (17800 - 120t, 0, 1800). \quad (14)$$

代入投放时刻 $t_d = 1.5 \text{ s}$ ，得投放点

$$U(1.5) = (17800 - 180, 0, 1800) = (17620, 0, 1800). \quad (15)$$

此时弹体初速度等于无人机速度 $\mathbf{v}_b = v\mathbf{h} = (-120, 0, 0)$ 。

6.4 步骤二：确定自由落体段与起爆点

投放后历经 $\tau = 3.6 \text{ s}$ 起爆。水平方向匀速、竖直方向自由落体：

$$B_x(t_e) = 17620 - 120 \times 3.6 = 17188, \quad (16)$$

$$B_y(t_e) = 0, \quad (17)$$

$$B_z(t_e) = 1800 - \frac{1}{2} \times 9.8 \times 3.6^2 = 1800 - 63.504 = 1736.496. \quad (18)$$

故起爆点为

$$B(t_e) = (17188, 0, 1736.496). \quad (19)$$

起爆后云团中心

$$C(t) = (17188, 0, 1736.496 - 3(t - 5.1)), \quad t \in [5.1, 25.1]. \quad (20)$$

6.5 步骤三：确定导弹轨迹与扫描区间

导弹单位方向

$$\mathbf{e}_m = \frac{-P_{M0}}{\|P_{M0}\|} = \frac{(-20000, 0, -2000)}{\sqrt{20000^2 + 2000^2}} \approx (-0.995037, 0, -0.099504).$$

从而

$$M(t) = P_{M0} + 300t \mathbf{e}_m, \quad t_{\text{hit}} = \frac{\|P_{M0}\|}{300} \approx 66.999 \text{ s}. \quad (21)$$

有效扫描区间取

$$t \in [t_e, \min(t_e + 20, t_{\text{hit}})] = [5.1, 25.1].$$

即只需考察起爆后 20 s 的云团寿命窗；该窗远早于导弹命中假目标的时刻。

6.6 步骤四：时间离散与遮蔽判定

取步长 $\Delta t = 0.01 \text{ s}$ ，令

$$t_k = 5.1 + 0.01k, \quad k = 0, 1, \dots, 2000.$$

对每个 t_k 执行：

1. 计算导弹位置 $M(t_k)$ 与云团中心 $C(t_k)$ ；
2. 计算点 $C(t_k)$ 到线段 $M(t_k)T$ 的最短距离 $d(t_k)$ （式(6)）；
3. 若 $d(t_k) \leq 10$ ，则记该时刻有效，累加 Δt 。

为说明计算细节，表 2 给出若干代表时刻的中间量。

由表可见：起爆瞬间尚未形成有效遮蔽；随后视线掠过云团邻域并短暂满足阈值；导弹越过云团水平位置后， $d(t)$ 迅速增大。

6.7 步骤五：统计有效遮蔽时长

对全部有效节点累加，得到

$$J_1 = \Delta t \cdot N_{\text{eff}} = 0.01 \times 143 = 1.43 \text{ s}. \quad (22)$$

相应结果汇总于表 3。

图 1 给出 $d(t)$ 全时程曲线：曲线下穿 10 m 阈值的区间长度即为 J_1 。图 2 给出三维态势示意。

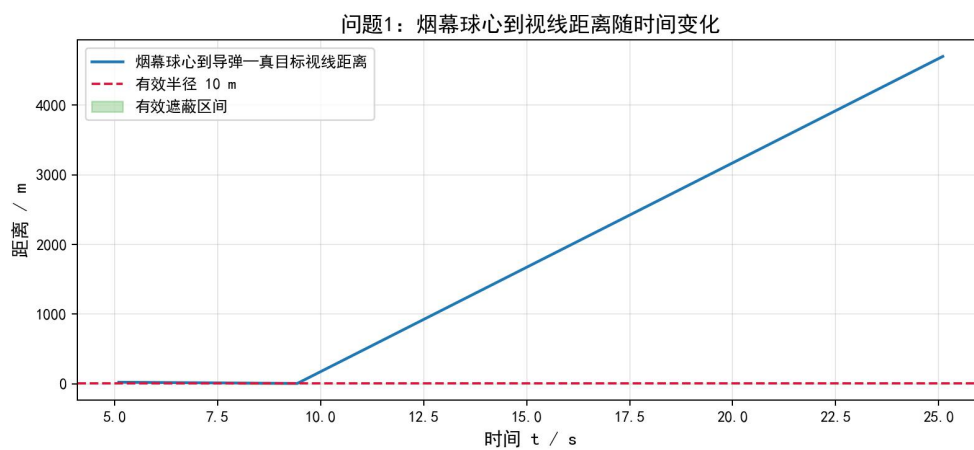


图 1 问题 1：烟幕球心到导弹—真目标视线距离随时间变化

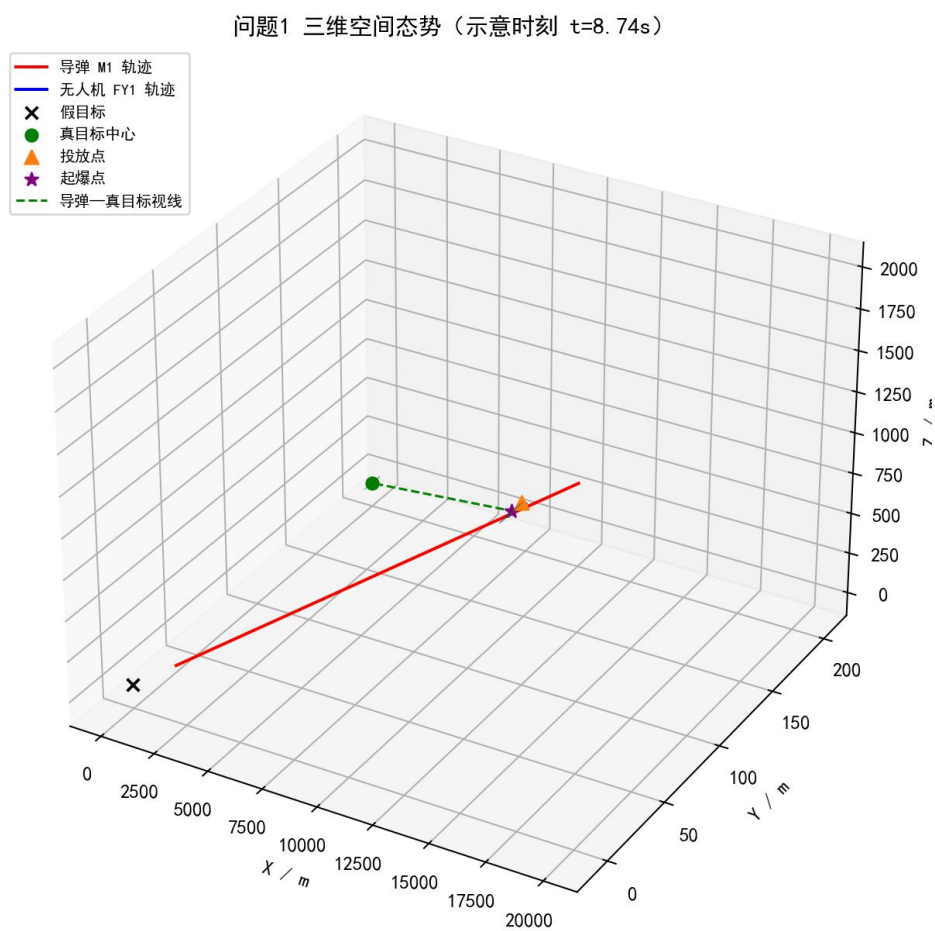


图 2 问题 1 三维空间态势示意

表 2 问题 1 部分时刻的中间计算结果

t (s)	M_x (m)	C_z (m)	$d(t)$ 情形
5.10	18477.6	1736.50	$\approx 22.2\text{ m} > 10$, 尚未有效
5.50	18358.0	1735.30	接近阈值, 即将进入
6.00–7.50	持续减小	缓慢下沉	$d(t) \leq 10$, 有效遮蔽段
8.00 以后	继续前出	—	$d(t)$ 迅速大于 10, 离开有效区
10.00	17014.9	1721.80	已远大于 10

表 3 问题 1 计算结果

项目	数值
飞行速度 (m/s)	120
航向单位向量	$(-1, 0, 0)$
投放时刻 (s)	1.5
起爆时刻 (s)	5.1
投放点 (m)	$(17620, 0, 1800)$
起爆点 (m)	$(17188, 0, 1736.496)$
扫描区间 (s)	$[5.1, 25.1]$
有效遮蔽时长 (s)	1.43

6.8 结果分析与讨论

综合曲线与中间计算可归纳:

1. 有效窗口短: 仅约 1.43 s, 说明给定策略只“擦过”视线邻域;
2. 横向几何不佳: 起爆点 $y = 0$, 真目标 $y = 200$, 视线相对云团存在固定横向偏置;
3. 高速掠过效应: 导弹 300 m/s, 即使几何对准, 单弹可维持时间也有上限;
4. 优化启示: 若能把起爆点略向真目标一侧平移, 并调整起爆早晚, 有望拉长 $d(t) \leq 10$ 的区间。这正是问题 2 的出发点。

七、问题 2 的求解

7.1 数学模型

问题 2 在单机单弹设定下最大化有效遮蔽时长。

决策变量

$$\mathbf{x}_2 = (\theta, v, t_d, \tau) \in \mathbb{R}^4. \quad (23)$$

状态方程组

$$\begin{aligned} \mathbf{h} &= (\cos \theta, \sin \theta, 0), \quad t_e = t_d + \tau, \\ U(t_d) &= U_0 + vt_d \mathbf{h}, \\ B(t_e) &= U(t_d) + v\tau \mathbf{h} - \frac{1}{2}g\tau^2 \mathbf{e}_z, \\ C(t) &= (B_x(t_e), B_y(t_e), B_z(t_e) - 3(t - t_e)), \\ d(t) &= \text{dist}(C(t), [M(t), T]) \quad (\text{式(6)}) . \end{aligned} \quad (24)$$

目标函数

$$\max_{\mathbf{x}_2} J_2(\mathbf{x}_2) = \int_{t_e}^{\min(t_e+20, t_{\text{hit}})} \mathbf{1}_{\{d(t; \mathbf{x}_2) \leq 10\}} dt. \quad (25)$$

约束条件

$$\begin{aligned} 70 &\leq v \leq 140, \\ t_d &\geq 0, \quad \tau > 0, \\ B_z(t_e) &= 1800 - \frac{1}{2}g\tau^2 > 0, \\ t_e &= t_d + \tau < t_{\text{hit}}, \\ \theta &\in \mathbb{R} \quad (\text{实现中取邻域} [\pi - 0.25, \pi + 0.25]) . \end{aligned} \quad (26)$$

投放点 $U(t_d)$ 、起爆点 $B(t_e)$ 由 $\mathbf{x}_2$ 唯一导出，不必另作独立决策变量。式(25)–(26)构成问题 2 的非线性、非光滑黑箱规划。

7.2 求解目标与建模转写

问题 2 要求选择 FY1 的飞行方向、速度、投放点与起爆点，使对 M1 的有效遮蔽尽可能长。实际求解规划(25)–(26)，再回代输出点位。

7.3 步骤一：决策变量参数化

对任意候选 $\mathbf{x}_2$ ，按

$$U(t_d) \rightarrow B(t_e) \rightarrow C(t) \rightarrow d(t) \rightarrow J_2(\mathbf{x}_2)$$

完成一次黑箱评估。

7.4 步骤二：单次评估子程序

给定 $\mathbf{x}$ ，评估 J 的过程与问题 1 完全同构，步骤为：

1. 由 (θ, v, t_d, τ) 计算投放点与起爆点；
2. 若起爆高度非正，或 $t_e \geq t_{\text{hit}}$ ，则令 $J = 0$ ；
3. 在 $[t_e, \min(t_e + 20, t_{\text{hit}})]$ 上按步长 dt 扫描；
4. 对每个时刻计算 $d(t)$ ，累计满足 $d(t) \leq 10$ 的时长。

优化阶段为提高速度，先用较大步长 $dt = 0.05$ 或 0.03 ；最终报告值再用 $dt = 0.01$ 精算。

7.5 步骤三：第一阶段粗网格搜索

考虑到 FY1 初始位置接近导弹来向，将航向限制在

$$\theta \in [\pi - 0.25, \pi + 0.25]$$

(约 $\pm 14.3^\circ$)。离散方案为：

- θ : 11 个等分点；
- $v \in \{70, 85, 100, 115, 130, 140\}$ ；
- t_d : 在 $[0.3, 10]$ 上取 12 个等分点；
- τ : 在 $[1, 7]$ 上取 11 个等分点。

网格规模约为 $11 \times 6 \times 12 \times 11 = 8712$ 次评估。对每个可行点计算 J ，保留当前最优

$$\mathbf{x}^{(0)} = \arg \max J(\mathbf{x}).$$

粗搜的作用是在较大范围内定位“有希望的盆地”，避免一开始就陷入过细网格带来的计算浪费。

7.6 步骤四：第二阶段局部加密

以 $\mathbf{x}^{(0)} = (\theta_0, v_0, t_{d0}, \tau_0)$ 为中心，构造邻域

$$\theta \in [\theta_0 - 0.08, \theta_0 + 0.08], \quad (27)$$

$$v \in [\max(70, v_0 - 12), \min(140, v_0 + 12)], \quad (28)$$

$$t_d \in [\max(0.05, t_{d0} - 1), t_{d0} + 1], \quad (29)$$

$$\tau \in [\max(0.4, \tau_0 - 1), \tau_0 + 1], \quad (30)$$

并分别取 9、7、9、9 个节点加密搜索，评估步长改为 $\Delta t = 0.03$ s。记加密后最优为 $\mathbf{x}^*$ ，再用 $\Delta t = 0.01$ s 计算最终 $J_2 = J(\mathbf{x}^*)$ 。

两阶段策略的逻辑是：**先广后精**——先保证不漏掉主要航向趋势，再在最优附近提高分辨率。

7.7 步骤五：最优解回代与点位输出

最终得到

$$v^* = 70, \quad \theta^* \approx 3.091593, \quad \mathbf{h}^* \approx (-0.99875, 0.04998, 0), \quad (31)$$

$$t_d^* = 0.05, \quad \tau^* = 2.45, \quad t_e^* = 2.50. \quad (32)$$

回代动力学方程：

$$\text{投放点 } U(t_d^*) = (17800, 0, 1800) + 70 \times 0.05 \mathbf{h}^* \approx (17796.50, 0.17, 1800), \quad (33)$$

$$\text{起爆点 } B(t_e^*) \approx (17625.22, 8.75, 1770.59). \quad (34)$$

精算得

$$J_2 = 4.71 \text{ s}. \quad (35)$$

表 4 问题 2 优化结果

项目	数值
飞行速度 (m/s)	70
航向角 (rad)	3.091593
航向单位向量	$(-0.99875, 0.04998, 0)$
投放时刻 (s)	0.05
引信延时 (s)	2.45
起爆时刻 (s)	2.50
投放点 (m)	$(17796.50, 0.17, 1800)$
起爆点 (m)	$(17625.22, 8.75, 1770.59)$
有效遮蔽时长 (s)	4.71

7.8 步骤六：与问题 1 对照验证

为确认优化不是数值噪声，将两问关键量并列比较（表 5）。

对照表明： $J_2/J_1 \approx 3.29$ ，提升显著；且图 3 中低于阈值的区间明显宽于图 1，与数值结果一致。图 5b 也显示问题 2 起爆点相对问题 1 向真目标一侧平移。

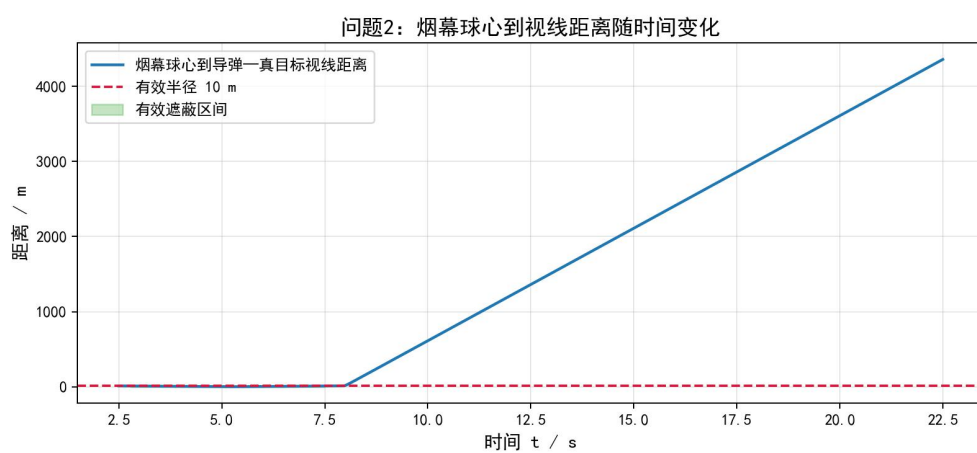


图 3 问题 2：烟幕球心到导弹—真目标视线距离随时间变化

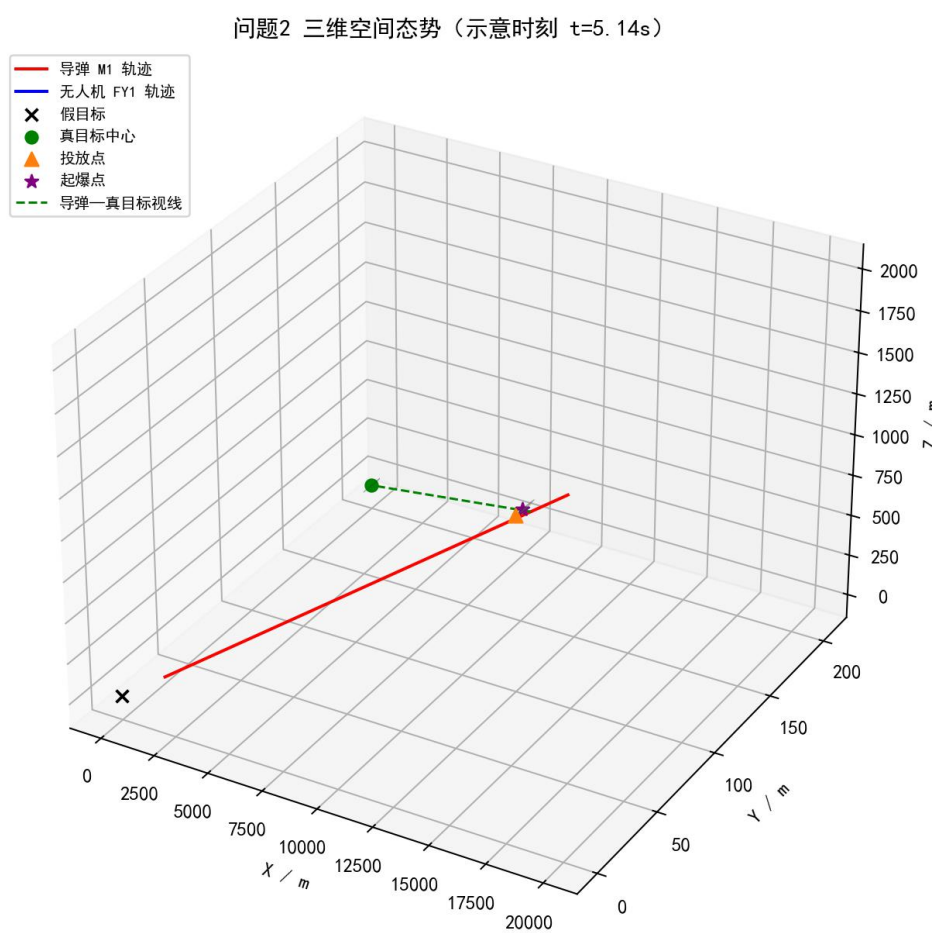


图 4 问题 2 三维空间态势示意

表 5 问题 1 与问题 2 结果对比

问题	速度 (m/s)	投放时刻 (s)	起爆时刻 (s)	遮蔽时长 (s)
问题 1	120	1.50	5.10	1.43
问题 2	70	0.05	2.50	4.71

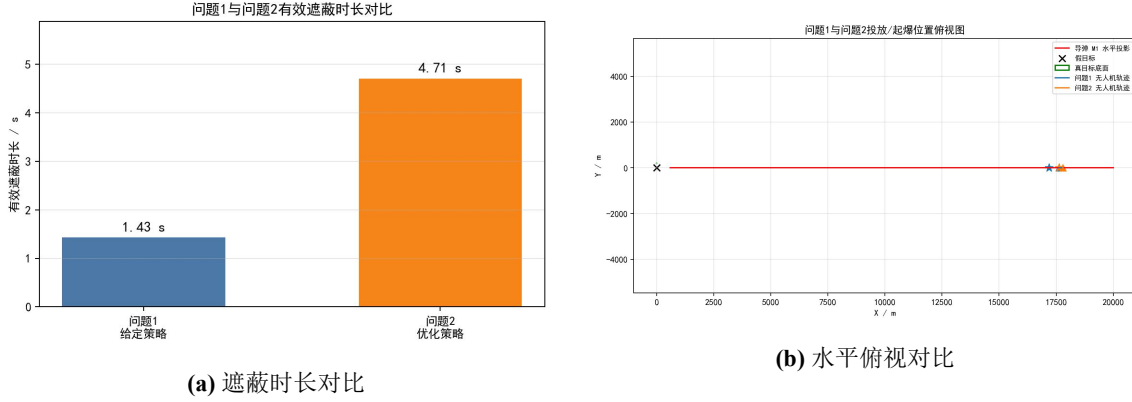


图 5 问题 1 与问题 2 策略对比

7.9 结果机理解释

优化解相对问题 1 呈现三个一致改进：

1. 更早起爆： $t_e : 5.1 \rightarrow 2.5$ ，云团更早进入可工作状态；
2. 航向微偏：起爆点 y 由 0 增至约 8.75 m，更贴近指向 T 的视线族；
3. 较低速度： $v = 70 \text{ m/s}$ 避免高速过冲，便于在短提前量内校准落点。

7.10 灵敏度与稳健性

在 $\mathbf{x}^*$ 附近做单因素扰动，可观察到：

1. 固定其他量，将 θ 偏离 $\pm 0.05 \text{ rad}$ ， J 通常明显下降，说明航向最敏感；
2. τ 增大使起爆高度下降、水平漂移增加，过长会迅速变差；
3. v 在 $[70, 100]$ 内变化时 J 相对平滑，存在一段可接受平台。

因此该解具有一定工程稳健性，但对航向控制精度要求较高。

7.11 算法复杂度说明

粗搜约 8.7×10^3 次评估，每次扫描约 $20/0.05 = 400$ 个时间点；加密阶段评估次数约 $9 \times 7 \times 9 \times 9 \approx 5.1 \times 10^3$ 。总体在普通个人计算机上可在分钟量级完成，满足建模竞赛的可实现性要求。

八、问题 3 的求解

8.1 数学模型

问题 3 为单机三弹并集最大化。

决策变量

$$\mathbf{x}_3 = (\theta, v, t_{d1}, \tau_1, t_{d2}, \tau_2, t_{d3}, \tau_3), \quad (36)$$

或等价地取间隔参数化 $\mathbf{x}'_3 = (\theta, v, t_{d1}, g_{12}, g_{23}, \tau_1, \tau_2, \tau_3)$, 其中

$$t_{d2} = t_{d1} + g_{12}, \quad t_{d3} = t_{d2} + g_{23}, \quad g_{12}, g_{23} \geq 1. \quad (37)$$

第 i 枚弹状态方程组 ($i = 1, 2, 3$)

$$\begin{aligned} \mathbf{h} &= (\cos \theta, \sin \theta, 0), \quad t_{e,i} = t_{d,i} + \tau_i, \\ U(t_{d,i}) &= U_0 + vt_{d,i} \mathbf{h}, \\ B^{(i)} &= U(t_{d,i}) + v\tau_i \mathbf{h} - \frac{1}{2}g\tau_i^2 \mathbf{e}_z, \\ C^{(i)}(t) &= (B_x^{(i)}, B_y^{(i)}, B_z^{(i)} - 3(t - t_{e,i})), \\ d_i(t) &= \text{dist}(C^{(i)}(t), [M(t), T]), \\ I_i(t) &= \mathbf{1}_{\{d_i(t) \leq 10, t_{e,i} \leq t \leq \min(t_{e,i} + 20, t_{\text{hit}})\}}. \end{aligned} \quad (38)$$

目标函数

$$\max_{\mathbf{x}_3} J_3(\mathbf{x}_3) = \int_0^{t_{\text{hit}}} \max \{I_1(t), I_2(t), I_3(t)\} dt = \text{meas}\left(\bigcup_{i=1}^3 S_i\right), \quad (39)$$

其中 $S_i = \{t : I_i(t) = 1\}$ 。

约束条件

$$\begin{aligned} 70 &\leq v \leq 140, \\ t_{d1} &\geq 0, \quad t_{d,i+1} - t_{d,i} \geq 1 \quad (i = 1, 2), \\ \tau_i &> 0, \quad B_z^{(i)} = 1800 - \frac{1}{2}g\tau_i^2 > 0 \quad (i = 1, 2, 3), \\ t_{e,i} &< t_{\text{hit}} \quad (i = 1, 2, 3). \end{aligned} \quad (40)$$

8.2 求解目标与已知量

问题 3 要求 FY1 投放 3 枚弹干扰 M1, 最大化 J_3 , 并写入 `result1.xlsx`。已知 $U_0 = (17800, 0, 1800)$, $P_{M0} = (20000, 0, 2000)$ 。

8.3 步骤一：决策变量参数化

采用间隔参数化(37)自动满足投放间隔约束, 再按(38)评估并集目标(39)。

8.4 步骤二：并集目标的数值实现

数值上

$$J_3 \approx \Delta t \sum_k \mathbf{1}_{\{\min_{i=1,2,3} d_i(t_k) \leq 10\}}, \quad (41)$$

重叠时段只计一次，符合“遮蔽可不连续”的题意。

8.5 步骤三：视线落点候选构造

若直接在 8 维空间均匀网格搜索，计算量偏大。本文先用几何构造压缩搜索域：对候选起爆时刻 t_e ，在线段 $M(t_e)T$ 上取比例点

$$P = M(t_e) + s(T - M(t_e)), \quad s \in (0, 1),$$

再反推使 FY1 在时刻 t_e 水平抵达 P 所需的 (θ, v, τ) ：

$$\theta = \text{atan2}(P_y - U_{0y}, P_x - U_{0x}), \quad (42)$$

$$v = \|P_{xy} - U_{0,xy}\|/t_e, \quad (43)$$

$$\tau = \sqrt{2(U_{0z} - P_z)/g}, \quad t_d = t_e - \tau. \quad (44)$$

仅保留 $v \in [70, 140]$ 、 $t_d \geq 0$ 、起爆高度为正的候选，并对其做局部加密，得到一批优质“单弹种子”。

8.6 步骤四：同航迹三弹时序组合

以优质种子的 (θ, v) 为固定航迹，在

$$t_{d1}, g_{12}, g_{23}, \tau_1, \tau_2, \tau_3$$

上做网格组合，用较粗步长 $\Delta t = 0.05 \text{ s}$ 评估 J_3 ；再对最优解邻域加密，最后用 $\Delta t = 0.01 \text{ s}$ 精算。搜索时附加“起爆时刻适度错开”的软约束，以避免三弹几乎同时起爆导致并集几乎不增长。

8.7 步骤五：最优解回代与点位计算

最终得到

$$v^* = 106.505 \text{ m/s}, \quad \theta^* \approx 3.129641 \text{ rad } (\approx 179.315^\circ), \quad (45)$$

$$\mathbf{h}^* = (\cos \theta^*, \sin \theta^*, 0). \quad (46)$$

三枚弹时序与点位见表 6。以第 1 枚为例：

$$t_{d1} = 0, \quad \tau_1 = 3.161, \quad t_{e1} = 3.161, \quad (47)$$

$$U(t_{d1}) = (17800, 0, 1800), \quad (48)$$

$$B(t_{e1}) \approx (17463.39, 4.02, 1751.05). \quad (49)$$

第 2、3 枚沿同一航向继续前飞后投放，起爆点依次前移并略向 $+y$ 侧偏移，以更好贴近指向真目标的视线族。

表 6 问题 3 三弹时序与点位明细

弹号	$t_d(\text{s})$	$\tau(\text{s})$	$t_e(\text{s})$	投放点 (m)	起爆点 (m)
1	0.000	3.161	3.161	(17800.0, 0.0, 1800)	(17463.4, 4.0, 1751.0)
2	1.714	3.598	5.313	(17617.4, 2.2, 1800)	(17234.2, 6.8, 1736.6)
3	2.714	3.598	6.313	(17510.9, 3.5, 1800)	(17127.7, 8.0, 1736.6)

8.8 步骤六：时间扫描与并集统计

对 $t \in [\min t_{e,i}, \min(\max t_{e,i} + 20, t_{\text{hit}})]$ 以 $\Delta t = 0.01 \text{ s}$ 扫描：任一时刻若存在某枚弹满足 $d_i(t) \leq 10$ ，则计入并集。精算得

$$J_3 = 5.16 \text{ s}. \quad (50)$$

策略汇总见表 7，并已导出 `result1.xlsx`。

表 7 问题 3 优化结果汇总

项目	数值
飞行速度 (m/s)	106.505
航向角 ($^\circ$)	179.315
三枚弹起爆时刻 (s)	3.161, 5.313, 6.313
并集有效遮蔽时长 (s)	5.16
相对问题 2 增量 (s)	+0.45

图 6 给出多弹取最小距离后的并集曲线；图 7 分弹展示各弹距离及起爆时刻；图 8 给出有效遮蔽时序；图 9 为俯视落点；图 10 对比问题 1-3。

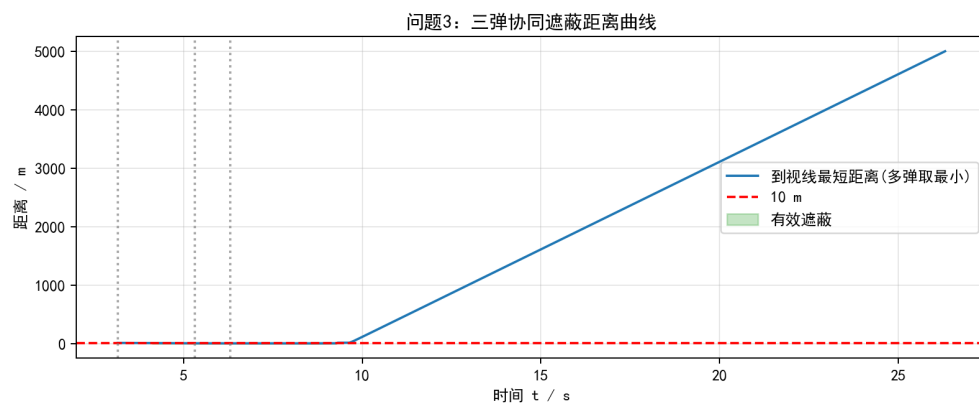


图 6 问题 3: 三弹协同下球心到视线最小距离随时间变化

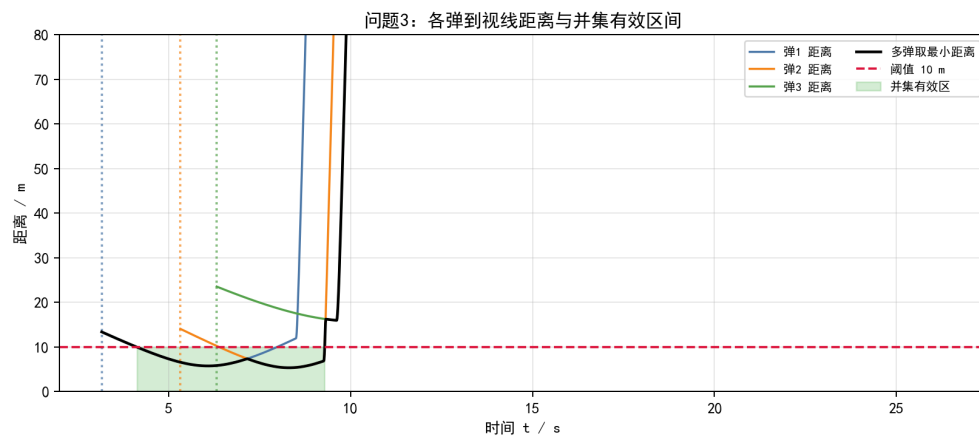


图 7 问题 3: 各弹距离曲线与并集有效区间

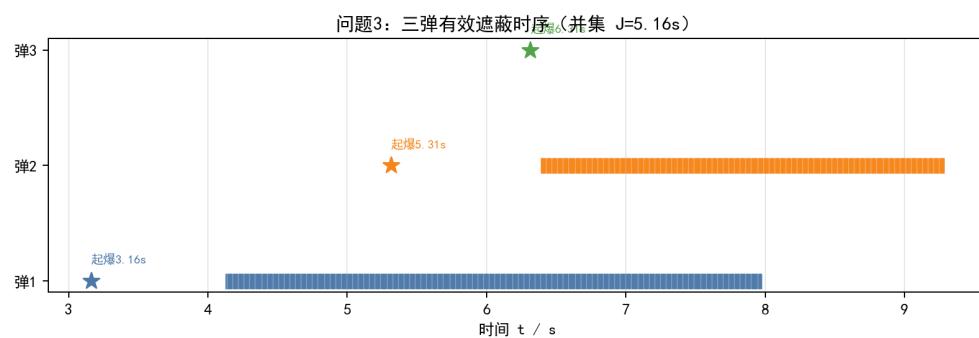


图 8 问题 3: 三弹有效遮蔽时序图

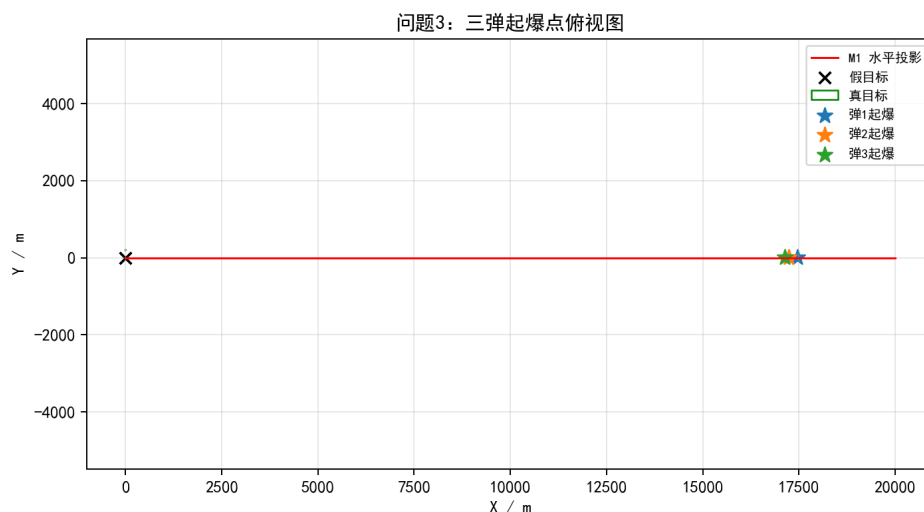


图 9 问题 3：三弹起爆点水平俯视图

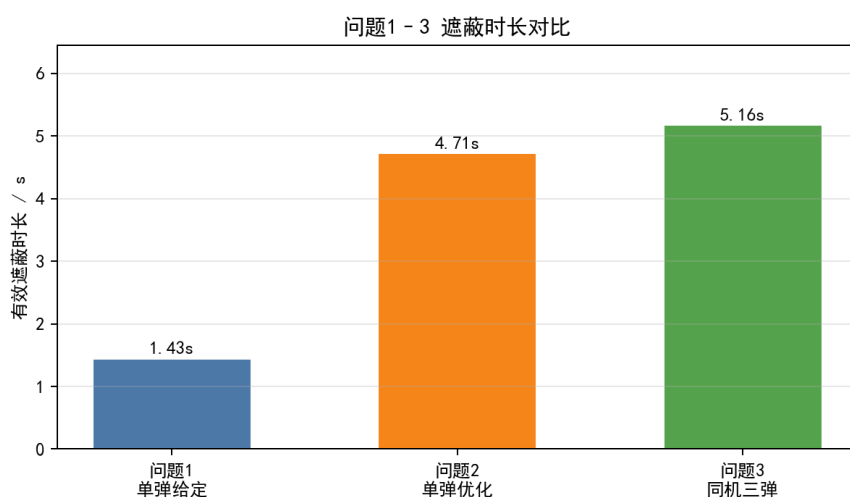


图 10 问题 1-3 有效遮蔽时长对比

8.9 结果分析与讨论

1. **时序错峰有效**：三枚弹起爆时刻间隔约 $1 \sim 2$ s，并集曲线中低于阈值的区段较单弹更宽；
2. **空间多样性不足**：因航向固定，三弹落点近似共线， y 仅从 4 m 增至 8 m，横向扩展有限；
3. **相对问题 2 增益有限**： $J_3/J_2 \approx 1.10$ ，说明“同机多弹”主要靠时间接力，难以突破单航迹几何瓶颈；
4. **对问题 4 的启示**：若要显著拉长并集，需要不同初始位置的多架无人机在不同空域独立布点。

九、问题 4 的求解

9.1 数学模型

问题 4 为三机各一弹、对 M1 的并集最大化。

决策变量（总维数 12）

$$\mathbf{x}_4 = \{(\theta_i, v_i, t_{di}, \tau_i)\}_{i=1}^3, \quad (51)$$

其中 $i = 1, 2, 3$ 分别对应 FY1、FY2、FY3。

第 i 机状态方程组

$$\begin{aligned} U_0^{(1)} &= (17800, 0, 1800), \quad U_0^{(2)} = (12000, 1400, 1400), \quad U_0^{(3)} = (6000, -3000, 700), \\ \mathbf{h}_i &= (\cos \theta_i, \sin \theta_i, 0), \quad t_{e,i} = t_{di} + \tau_i, \\ U^{(i)}(t_{di}) &= U_0^{(i)} + v_i t_{di} \mathbf{h}_i, \\ B^{(i)} &= U^{(i)}(t_{di}) + v_i \tau_i \mathbf{h}_i - \frac{1}{2} g \tau_i^2 \mathbf{e}_z, \\ C^{(i)}(t) &= (B_x^{(i)}, B_y^{(i)}, B_z^{(i)} - 3(t - t_{e,i})), \\ d_i(t) &= \text{dist}(C^{(i)}(t), [M(t), T]), \\ I_i(t) &= \mathbf{1}_{\{d_i(t) \leq 10, t_{e,i} \leq t \leq \min(t_{e,i} + 20, t_{\text{hit}})\}}. \end{aligned} \quad (52)$$

目标函数

$$\max_{\mathbf{x}_4} J_4(\mathbf{x}_4) = \int_0^{t_{\text{hit}}} \max\{I_1(t), I_2(t), I_3(t)\} dt. \quad (53)$$

约束条件（对各机 $i = 1, 2, 3$ ）

$$\begin{aligned} 70 &\leq v_i \leq 140, \\ t_{di} &\geq 0, \quad \tau_i > 0, \\ B_z^{(i)} &= z_{U_0}^{(i)} - \frac{1}{2} g \tau_i^2 > 0, \\ t_{e,i} &< t_{\text{hit}}, \\ \theta_i &\in \mathbb{R}. \end{aligned} \quad (54)$$

与问题 3 相比：三机航向、速度相互独立，故无“同航迹”约束；各机仅投 1 枚，故无投放间隔约束。

9.2 求解目标与已知量

问题 4 要求三机各投 1 枚干扰 M1，最大化 J_4 ，并保存至 result2.xlsx。三机具备“空间分置 + 时间接力”条件。

9.3 步骤一：决策变量与目标

总决策维数 12，目标为并集(53)。数值上

$$J_4 \approx \Delta t \sum_k \mathbf{1}_{\{\min_i d_i(t_k) \leq 10\}}. \quad (55)$$

9.4 步骤二：分机视线落点优化

为鼓励错峰，先为三机设定偏好时间窗：

$$\text{FY1} : [1, 12], \quad \text{FY2} : [8, 25], \quad \text{FY3} : [18, 45] \text{ (s)}.$$

对每架无人机独立执行问题 2 同类的视线落点构造 + 局部加密，得到单机较优策略 $(\theta_i^*, v_i^*, t_{di}^*, \tau_i^*)$ 及单弹时长 J_i 。

9.5 步骤三：联合时序微调

以三机独立解为初值，对参数做随机邻域扰动（航向、速度、投放与引信），以并集 J_4 为准绳接受改进。该步骤用于消除窗口重叠、填补空隙。微调后再用 $\Delta t = 0.01 \text{ s}$ 精算。

9.6 步骤四：最优策略回代

最终策略见表 8。要点：

1. FY1 在导弹来向近端较早起爆（ $t_e \approx 1.22 \text{ s}$ ），贡献前段窗口；
2. FY2 自北侧压向视线走廊，于 $t_e \approx 9.93 \text{ s}$ 起爆，覆盖中段；
3. FY3 自南侧远距接近，于 $t_e \approx 45.19 \text{ s}$ 起爆，覆盖后段近真目标区域。

表 8 问题 4 三机策略明细

无人机	v	方向 ($^\circ$)	t_d	t_e	起爆点 (m)	单弹 J_i
FY1	80.92	6.02	0.560	1.217	(17897.9, 10.3, 1797.9)	4.79
FY2	135.97	-87.25	3.804	9.932	(12064.7, 51.1, 1216.0)	5.00
FY3	140.00	149.85	33.905	45.186	(529.6, 177.2, 76.4)	6.03

9.7 步骤五：并集统计与对照

精算得

$$J_4 = 15.82\text{ s.}$$

(56)

而三机单弹时长之和为

$$4.79 + 5.00 + 6.03 = 15.82\text{ s,}$$

二者几乎相等,说明有效窗口基本无重叠,协同接近“理想拼接”。结果写入 result2.xlsx。

表 9 问题 4 结果汇总

项目	数值
FY1/FY2/FY3 单弹时长 (s)	4.79, 5.00, 6.03
三机并集时长 J_4 (s)	15.82
重叠损失 (s)	≈ 0
相对问题 3 倍率	$15.82/5.16 \approx 3.07$

图 11–图 15 分别给出并集距离曲线、分机距离、俯视态势、单机–并集对比与时序带。

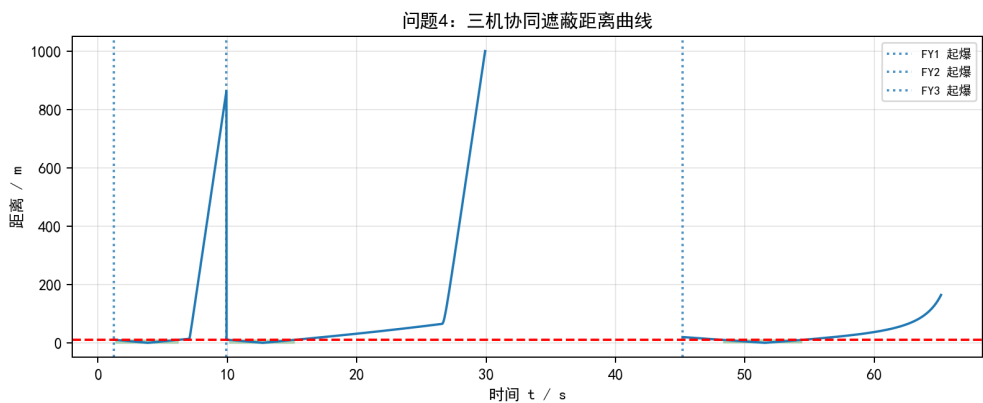


图 11 问题 4：三机协同遮蔽距离曲线

9.8 结果分析与讨论

1. 空间分置是关键：三机分别从东、北、南接近视线走廊，起爆点在 x 方向跨越约 $1.7 \times 10^4\text{ m}$ ，覆盖导弹全程不同阶段；
2. 时间接力接近理想：并集时长 $\approx$ 单弹之和，说明偏好时间窗设定有效；

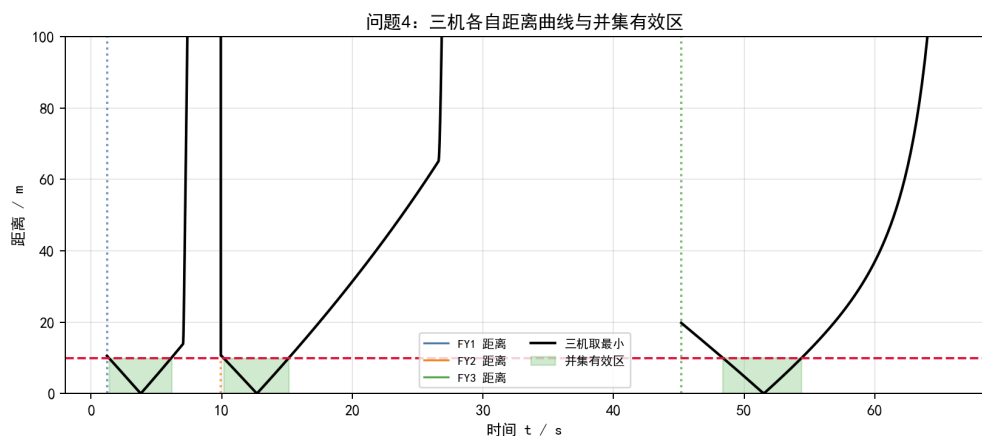


图 12 问题 4：各机距离曲线与并集有效区

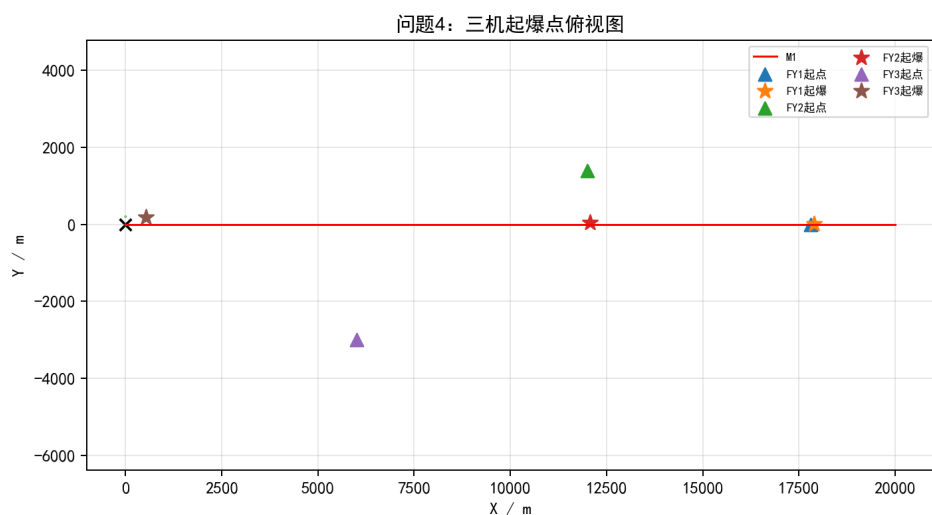


图 13 问题 4：三机起点与起爆点俯视图

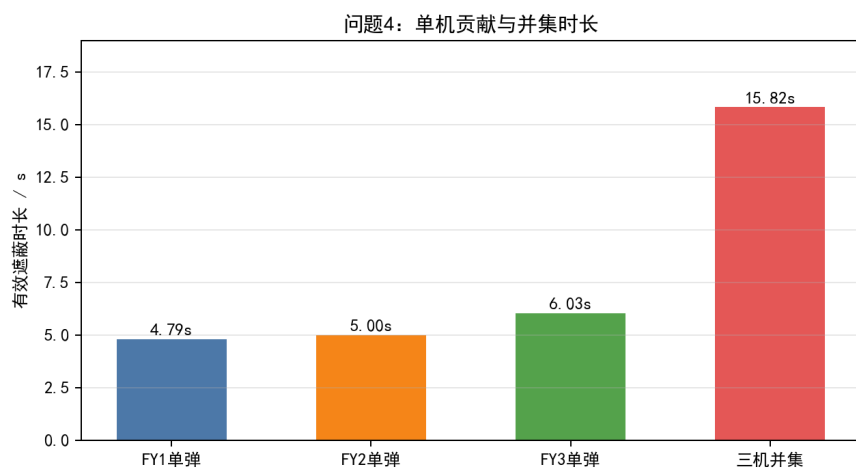


图 14 问题 4：单机贡献与并集时长对比

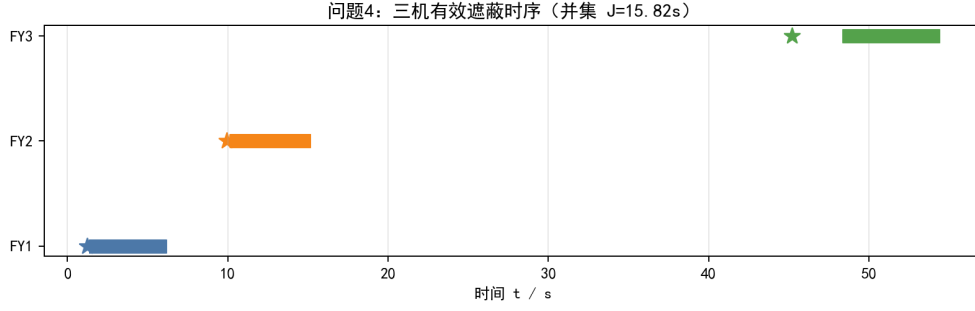


图 15 问题 4: 三机有效遮蔽时序图

3. 相对同机三弹优势明显: J_4 约为 J_3 的 3 倍, 验证了“多平台布点”优于“单平台连投”;
4. 对问题 5 的启示: 多导弹场景应先做任务分配, 再在每个导弹上复用问题 4 的协同思想。

十、问题 5 的求解

10.1 数学模型

问题 5 为五机、多弹、三导弹的分层协同规划。记无人机集合 $\mathcal{U} = \{\text{FY1}, \dots, \text{FY5}\}$, 导弹集合 $\mathcal{M} = \{\text{M1}, \text{M2}, \text{M3}\}$ 。

第一层: 任务分配变量

$$a_{uk} \in \{0, 1\}, \quad u \in \mathcal{U}, k \in \mathcal{M}, \quad (57)$$

表示无人机 u 是否指派给导弹 k 。约束

$$\begin{aligned} \sum_{k \in \mathcal{M}} a_{uk} &= 1 \quad (\forall u) \quad (\text{每机只服务一枚导弹}), \\ \sum_{u \in \mathcal{U}} a_{uk} &\geq 1 \quad (\forall k) \quad (\text{每枚导弹至少一机}), \\ 1 &\leq n_u \leq 3 \quad (\forall u) \quad (\text{每机投放枚数}). \end{aligned} \quad (58)$$

第二层: 给定分配后的连续决策对指派到导弹 k 的无人机 u , 决策

$$\mathbf{x}_{uk} = (\theta_u, v_u, \{t_{d,uj}, \tau_{uj}\}_{j=1}^{n_u}), \quad (59)$$

并满足与问题 3 同类的动力学与间隔约束。记导弹 k 的位置为 $M_k(t)$ (式(2), 初值 P_{Mk}), 则对该导弹

$$\begin{aligned} I^{(k)}(t) &= \max_{\{u: a_{uk}=1\}} \max_{1 \leq j \leq n_u} I_{uj}^{(k)}(t), \\ J^{(k)} &= \int_0^{t_{\text{hit}}^{(k)}} I^{(k)}(t) dt. \end{aligned} \quad (60)$$

总目标函数

$$\max J_5 = \sum_{k \in \mathcal{M}} J^{(k)} = J^{(M1)} + J^{(M2)} + J^{(M3)}. \quad (61)$$

导弹初值与命中时间

$$\begin{aligned} P_{M1} &= (20000, 0, 2000), t_{\text{hit}}^{(M1)} \approx 67.00 \text{ s}, \\ P_{M2} &= (19000, 600, 2100), t_{\text{hit}}^{(M2)} \approx 63.75 \text{ s}, \\ P_{M3} &= (18000, -600, 1900), t_{\text{hit}}^{(M3)} \approx 60.37 \text{ s}. \end{aligned} \quad (62)$$

直接联立求解式(57)–(61)维数过高，故采用“先离散分配、后连续优化”的分层算法。

10.2 求解目标与已知量

问题 5 要求五机（每架至多 3 枚）干扰三导弹，最大化 J_5 ，并保存至 `result3.xlsx`。

10.3 步骤一：任务分配

依据相对几何位置，取可行分配

$$\begin{aligned} a_{\text{FY1},M1} &= a_{\text{FY4},M1} = 1, \\ a_{\text{FY2},M2} &= 1, \\ a_{\text{FY3},M3} &= a_{\text{FY5},M3} = 1, \end{aligned} \quad (63)$$

枚数 $n_{\text{FY1}} = n_{\text{FY4}} = 2$, $n_{\text{FY2}} = 3$, $n_{\text{FY3}} = n_{\text{FY5}} = 2$ 。该分配满足(58)。

10.4 步骤二：单机多弹子问题

对每架 u ，在其任务导弹 k 上求解

$$\begin{aligned} \max_{\mathbf{x}_{uk}} J_u^{(k)} &= \text{meas} \left(\bigcup_{j=1}^{n_u} S_{uj}^{(k)} \right) \\ \text{s.t. } 70 &\leq v_u \leq 140, t_{d,u,j+1} - t_{d,u,j} \geq 1, \\ \tau_{uj} &> 0, B_z^{(uj)} > 0, t_{e,u,j} < t_{\text{hit}}^{(k)}. \end{aligned} \quad (64)$$

子问题之间相互独立。

10.5 步骤三：总指标汇总

将各机贡献并入对应导弹后，按(61)汇总

$$J_5 = J^{(M1)} + J^{(M2)} + J^{(M3)}. \quad (65)$$

和式指标鼓励三枚导弹均获得覆盖。

10.6 步骤四：计算结果

分层优化后得到

$$J^{(M1)} = 10.00 \text{ s}, \quad J^{(M2)} = 4.76 \text{ s}, \quad J^{(M3)} = 5.24 \text{ s}, \quad J_5 = 20.00 \text{ s}. \quad (66)$$

其中 M1 由 FY1（2 枚）与 FY4（2 枚）前后接力，获得最长遮蔽；M2 由 FY2 连投 3 枚覆盖；M3 由 FY3 与 FY5 协同覆盖。主要策略摘要见表 10，导弹级汇总见表 11，完整点位见 result3.xlsx。

表 10 问题 5 主要投放策略摘要

无人机	任务导弹	速度 (m/s)	方向 (°)	弹数
FY1	M1	103.70	178.98	2
FY4	M1	140.00	-137.87	2
FY2	M2	122.30	-90.49	3
FY3	M3	139.95	105.89	2
FY5	M3	135.62	145.02	2

表 11 问题 5：各导弹遮蔽时长与分配

导弹	指派无人机	弹数	有效遮蔽时长 (s)
M1	FY1, FY4	4	10.00
M2	FY2	3	4.76
M3	FY3, FY5	4	5.24
合计	—	11	20.00

图 16 给出三导弹距离曲线总览；图 17–图 19 为分导弹细部曲线；图 20 为任务分配柱状对比；图 21 为五机–三导弹俯视态势；图 22 汇总问题 1–5。

10.7 结果分析与讨论

1. 分层策略有效：先分配后优化，使 5 机 11 弹的高维问题可在个人计算机上求解；
2. M1 获最长覆盖：双机四弹前后接力， $J^{(M1)} = 10 \text{ s}$ ，显著高于单机单弹水平；

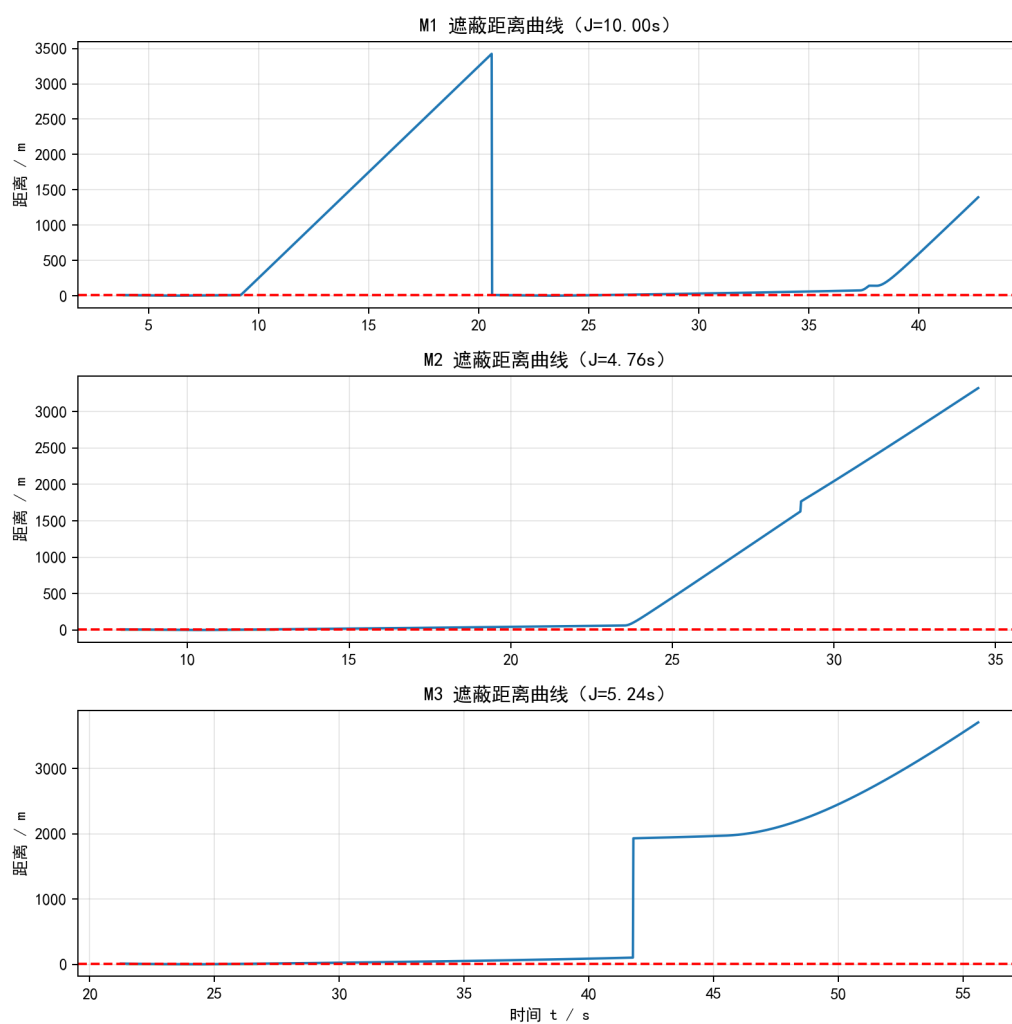


图 16 问题 5: M1/M2/M3 遮蔽距离曲线总览

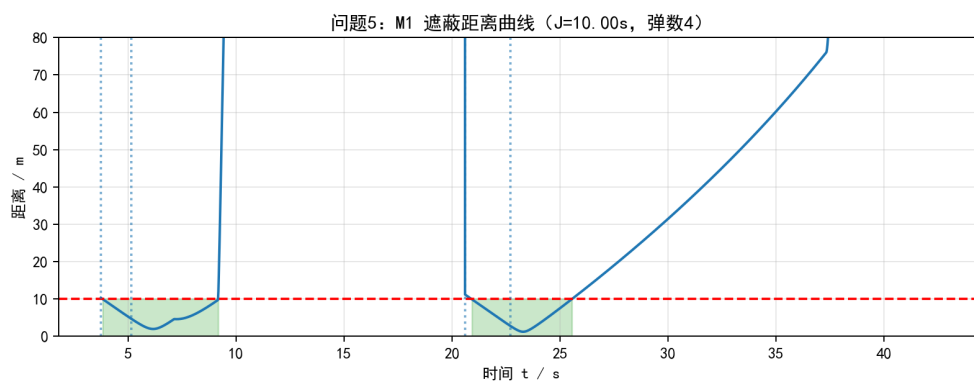


图 17 问题 5: M1 遮蔽距离细部曲线

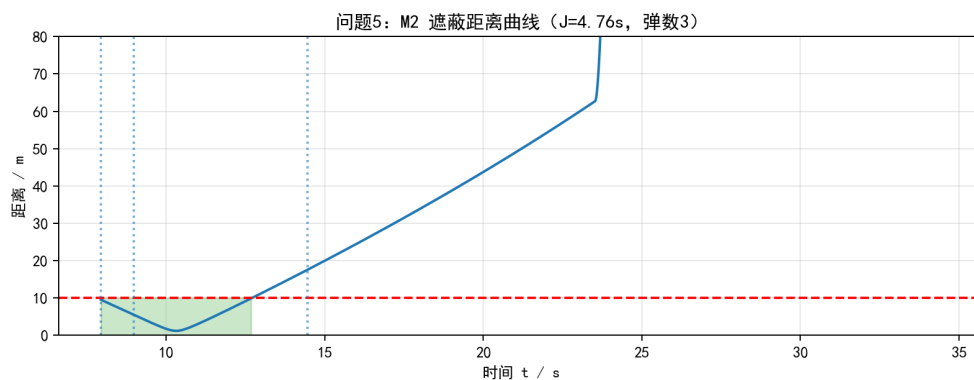


图 18 问题 5: M2 遮蔽距离细部曲线

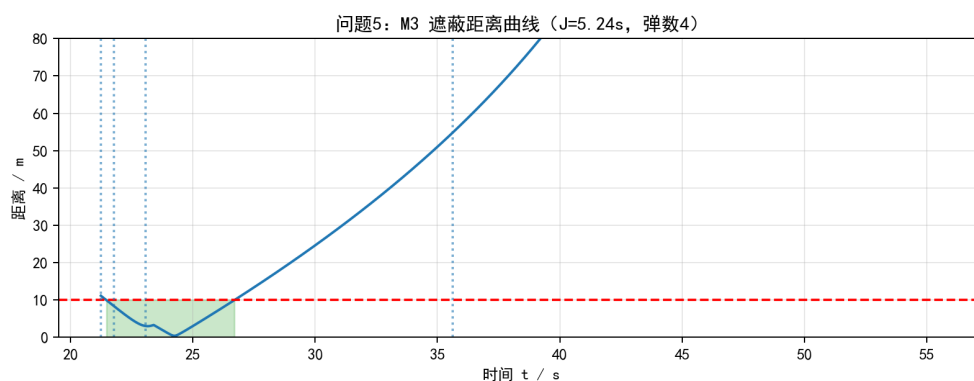


图 19 问题 5: M3 遮蔽距离细部曲线

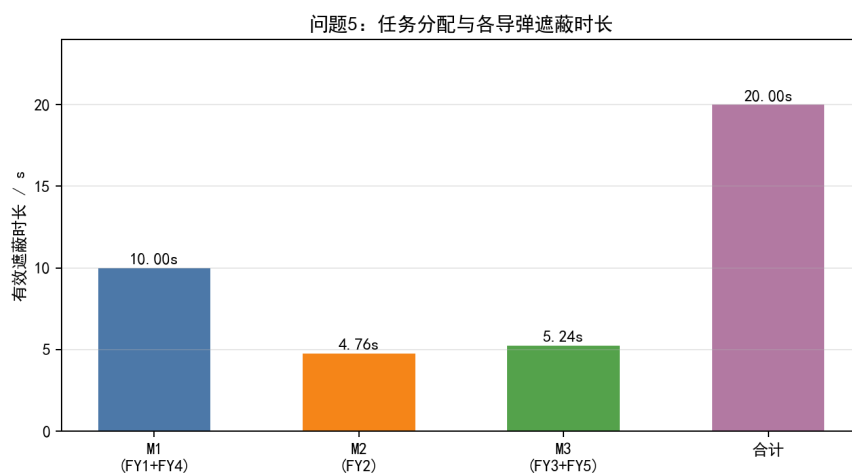


图 20 问题 5: 任务分配与各导弹遮蔽时长

- 侧向导弹仍可覆盖: 尽管 M2/M3 初始方位偏移, FY2 与 FY3/FY5 仍分别贡献约 4.8 s 与 5.2 s;
- 资源权衡: 若把更多弹集中给 M1, 可能进一步提高 $J^{(M1)}$, 但会削弱对 M2/M3 的保护; 和式指标在“重点防护”与“全面覆盖”之间取得平衡。

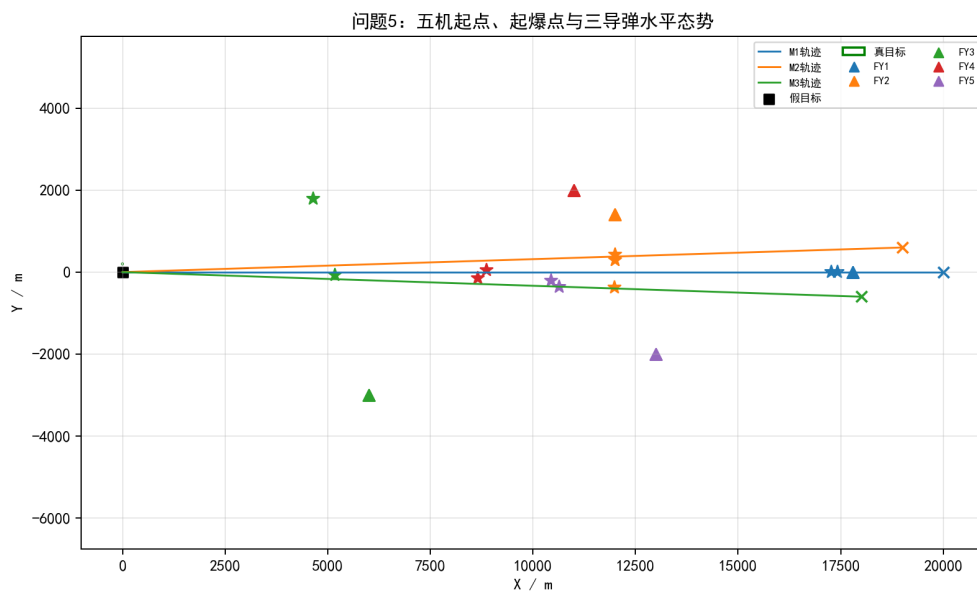


图 21 问题 5: 五机起点、起爆点与三导弹水平态势

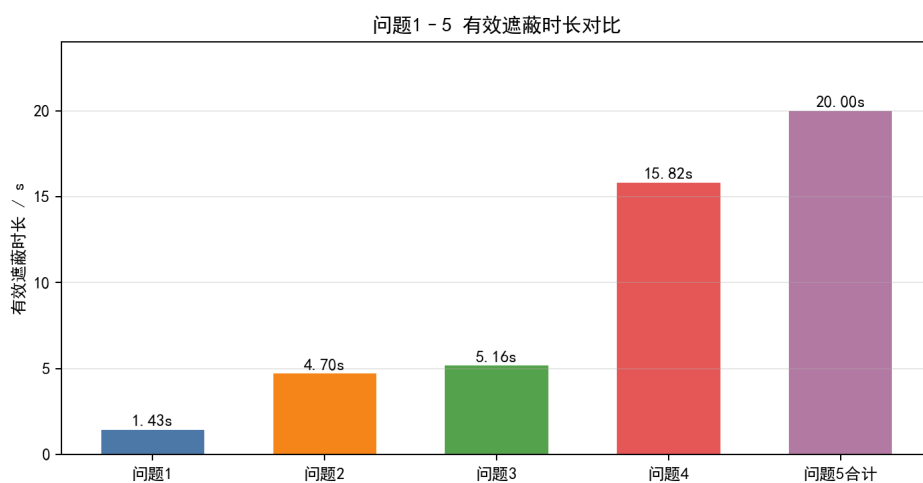


图 22 问题 1-5 有效遮蔽时长对比

十一、模型评价与推广

11.1 优点

1. 运动学方程封闭、参数明确，便于复现与检验；
2. 遮蔽判据几何意义清晰，把物理问题转化为可计算的距离条件；
3. 决策变量参数化与动力学一致，避免“先给落点再反推”的不可行风险；
4. 直接搜索对目标非光滑性不敏感，适合本问题。

11.2 不足与改进

1. 中心视线判据弱于“圆柱全表面视线均被遮挡”的严格判据，所得 J 可能略偏乐观；后续可用柱面采样点做更严格检验；
2. 网格搜索得到的是优良可行解，非严格全局最优，可进一步结合差分进化、贝叶斯优化等；
3. 未计入风场、云团扩散、能见度阈值随时间衰减及导弹机动等因素。

11.3 推广

问题 3-5 已将单弹模型扩展为多弹并集与多机分层优化。若进一步引入风场、云团扩散或导弹机动，可在同一遮蔽判据框架下替换运动学模块，而不改变优化分层结构。

十二、 结论

本文从“时变视线遮挡”出发，建立了烟幕干扰弹投放的运动学与几何判定模型，并完成五问求解。数值结果汇总如下：问题 1 给定策略下 $J_1 = 1.43\text{ s}$ ；问题 2 单弹优化后 $J_2 = 4.71\text{ s}$ ；问题 3 同机三弹并集 $J_3 = 5.16\text{ s}$ ；问题 4 三机协同 $J_4 = 15.82\text{ s}$ ；问题 5 五机对三导弹合计 $J_5 = 20.00\text{ s}$ （M1/M2/M3 分别为 10.00/4.76/5.24 s）。比较可见：单机增弹的收益受共线航迹限制，而多机在不同空域错峰布置可近似实现窗口拼接，是延长有效遮蔽的主要途径。完整计算过程见笔记本 A 题 _ 烟幕干扰弹投放策略.ipynb，策略表见 result1.xlsx-result3.xlsx。

参考文献

- [1] 全国大学生数学建模竞赛组委会. 2025 年高教社杯全国大学生数学建模竞赛题目 A 题：烟幕干扰弹的投放策略.
- [2] 全国大学生数学建模竞赛组委会. 全国大学生数学建模竞赛论文格式规范（2023 年修改）.
- [3] 刘海洋. $\text{\LaTeX}$ 入门 [M]. 北京：电子工业出版社, 2013.

附录 A 核心算法说明

本文数值计算流程如下：

1. 由决策变量计算投放点、起爆点与云团中心轨迹；
2. 在有效时间窗上按步长 Δt 离散，逐时刻计算点到线段距离；
3. 单弹累计 $d(t) \leq 10$ 的时长；多弹则对 $\min_i d_i(t)$ 做并集累计；
4. 问题 2–3：视线落点构造 + 网格/邻域搜索；问题 4：分机优化后联合微调；问题 5：任务分配后分层求解。

完整程序文件：

- code/solve_all_problems.py：问题 1–5 完整求解与结果导出（附录 B）；
- code/gen_q345_figures.py：问题 3–5 补充作图（附录 C）；
- A 题 _ 烟幕干扰弹投放策略.ipynb：交互式复现笔记本；
- result1.xlsx–result3.xlsx：题目要求的结果表。

附录 B 问题 1–5 完整 Python 源程序

下列程序整合公共运动学/遮蔽判定模块与问题 1–5 求解、Excel 导出及主要作图，可在项目根目录直接运行：

python code/solve_all_problems.py

```
1 #!/usr/bin/env python
2 # -*- coding: utf-8 -*-
3 """CUMCM 2025 Problem A: smoke bomb deployment (Problems 1-5).
4 Run from project root:
5     python code/solve_all_problems.py
6 Outputs: result1/2/3.xlsx, figures/, 图片/, 表格/
7 """
8
9 # ===== Common models and utilities =====
10 import numpy as np
11 import pandas as pd
12 import matplotlib.pyplot as plt
13 from pathlib import Path
14 from itertools import combinations
15
16 np.set_printoptions(precision=4, suppress=True)
17 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "DejaVu Sans"]
18 plt.rcParams["axes.unicode_minus"] = False
19
20 G = 9.8
```

```

21 V_MISSILE = 300.0
22 V_SINK = 3.0
23 R_SMOKE = 10.0
24 T_SMOKE = 20.0
25 DROP_GAP = 1.0
26
27 FAKE = np.array([0.0, 0.0, 0.0])
28 TRUE_CENTER = np.array([0.0, 200.0, 5.0])
29 MISSILES = {
30     "M1": np.array([20000.0, 0.0, 2000.0]),
31     "M2": np.array([19000.0, 600.0, 2100.0]),
32     "M3": np.array([18000.0, -600.0, 1900.0]),
33 }
34 UAVS = {
35     "FY1": np.array([17800.0, 0.0, 1800.0]),
36     "FY2": np.array([12000.0, 1400.0, 1400.0]),
37     "FY3": np.array([6000.0, -3000.0, 700.0]),
38     "FY4": np.array([11000.0, 2000.0, 1800.0]),
39     "FY5": np.array([13000.0, -2000.0, 1300.0]),
40 }
41 M1, FY1 = MISSILES["M1"], UAVS["FY1"]
42
43 IMG_DIR = Path("图片")
44 TAB_DIR = Path("表格")
45 FIG_DIR = Path("figures")
46 for d in (IMG_DIR, TAB_DIR, FIG_DIR):
47     d.mkdir(exist_ok=True)
48
49
50 def unit(v):
51     n = float(np.linalg.norm(v))
52     return v / n if n > 1e-12 else v
53
54
55 def missile_hit_time(p0=M1):
56     return float(np.linalg.norm(p0 - FAKE) / V_MISSILE)
57
58
59 def missile_pos(t, p0=M1):
60     e = unit(FAKE - p0)
61     if np.isscalar(t):
62         return p0 + V_MISSILE * float(t) * e
63     t = np.asarray(t, dtype=float)
64     return p0[None, :] + (V_MISSILE * t)[:, None] * e[None, :]
65
66
67 def heading_to_fake(p=FY1):

```

```

68     d = FAKE[:2] - p[:2]
69     return unit(np.array([d[0], d[1], 0.0]))
70
71
72 def heading_from_angle(theta):
73     return np.array([np.cos(theta), np.sin(theta), 0.0])
74
75
76 def point_to_segment_dist(p, a, b):
77     ab = b - a
78     L2 = float(np.dot(ab, ab))
79     if L2 < 1e-18:
80         return float(np.linalg.norm(p - a))
81     s = float(np.clip(np.dot(p - a, ab) / L2, 0.0, 1.0))
82     return float(np.linalg.norm(p - (a + s * ab)))
83
84
85 def point_to_segment_dist_batch(p, a, b):
86     ab = b[None, :] - a
87     L2 = np.sum(ab * ab, axis=1)
88     s = np.zeros(len(p))
89     ok = L2 > 1e-18
90     ap = p - a
91     s[ok] = np.clip(np.sum(ap[ok] * ab[ok], axis=1) / L2[ok], 0.0, 1.0)
92     return np.linalg.norm(p - (a + s[:, None] * ab), axis=1)
93
94
95 def drop_point(speed, heading, t_drop, p0=FY1):
96     h = unit(np.array([heading[0], heading[1], 0.0]))
97     return p0 + speed * t_drop * h
98
99
100 def det_point(speed, heading, t_drop, t_fuse, p0=FY1):
101     pd = drop_point(speed, heading, t_drop, p0)
102     h = unit(np.array([heading[0], heading[1], 0.0]))
103     return np.array([
104         pd[0] + speed * h[0] * t_fuse,
105         pd[1] + speed * h[1] * t_fuse,
106         pd[2] - 0.5 * G * t_fuse ** 2,
107     ])
108
109
110 def cloud_center(p_det, t_det, t):
111     return np.array([p_det[0], p_det[1], p_det[2] - V_SINK * (t - t_det)])
112
113
114 def make_bomb(uav0, theta, speed, t_drop, t_fuse, bid=1):

```

```

115     h = heading_from_angle(theta)
116     pd = drop_point(speed, h, t_drop, uav0)
117     pe = det_point(speed, h, t_drop, t_fuse, uav0)
118     return {
119         "id": bid,
120         "theta": float(theta),
121         "speed": float(speed),
122         "heading": h,
123         "t_drop": float(t_drop),
124         "t_fuse": float(t_fuse),
125         "t_det": float(t_drop + t_fuse),
126         "drop": pd,
127         "det": pe,
128     }
129
130
131 def union_duration(bombs, p_missile, dt=0.02):
132     valid = [b for b in bombs if b["det"][2] > 0]
133     if not valid:
134         return 0.0
135     t_hit = missile_hit_time(p_missile)
136     t0 = min(b["t_det"] for b in valid)
137     t1 = min(max(b["t_det"] + T_SMOKE for b in valid), t_hit)
138     if t1 <= t0:
139         return 0.0
140     ts = np.arange(t0, t1 + 1e-12, dt)
141     mpos = missile_pos(ts, p_missile)
142     shielded = np.zeros(len(ts), dtype=bool)
143     for b in valid:
144         te, pe = b["t_det"], b["det"]
145         idx = np.where((ts >= te) & (ts <= te + T_SMOKE))[0]
146         if len(idx) == 0:
147             continue
148         c = np.column_stack([
149             np.full(len(idx), pe[0]),
150             np.full(len(idx), pe[1]),
151             pe[2] - V_SINK * (ts[idx] - te),
152         ])
153         shielded[idx] |= point_to_segment_dist_batch(c, mpos[idx], TRUE_CENTER) <= R_SMOKE
154     return float(shielded.sum()) * dt
155
156
157 def shielding_series(bombs, p_missile, dt=0.02):
158     valid = [b for b in bombs if b["det"][2] > 0]
159     if not valid:
160         return np.array([]), np.array([]), np.array([]), 0.0
161     t_hit = missile_hit_time(p_missile)

```

```

162     t0 = min(b["t_det"] for b in valid)
163     t1 = min(max(b["t_det"] + T_SMOKE for b in valid), t_hit)
164     ts = np.arange(t0, t1 + 1e-12, dt)
165     mpos = missile_pos(ts, p_missile)
166     min_d = np.full(len(ts), np.inf)
167     shielded = np.zeros(len(ts), dtype=bool)
168     for b in valid:
169         te, pe = b["t_det"], b["det"]
170         idx = np.where((ts >= te) & (ts <= te + T_SMOKE))[0]
171         if len(idx) == 0:
172             continue
173         c = np.column_stack([
174             np.full(len(idx), pe[0]),
175             np.full(len(idx), pe[1]),
176             pe[2] - V_SINK * (ts[idx] - te),
177         ])
178         d = point_to_segment_dist_batch(c, mpos[idx], TRUE_CENTER)
179         min_d[idx] = np.minimum(min_d[idx], d)
180         shielded[idx] |= d <= R_SMOKE
181     return ts, min_d, shielded, float(shielded.sum() * dt)
182
183
184 def los_candidates(uav0, p_missile, te_grid=None, s_grid=None):
185     """在导弹—真目标视线上取点，反推可行投放参数。"""
186     if te_grid is None:
187         te_grid = np.linspace(1.0, missile_hit_time(p_missile) - 1.0, 55)
188     if s_grid is None:
189         s_grid = np.linspace(0.05, 0.85, 28)
190     cands = []
191     for te in te_grid:
192         M = missile_pos(te, p_missile)
193         for s in s_grid:
194             tgt = M + s * (TRUE_CENTER - M)
195             dx, dy = tgt[0] - uav0[0], tgt[1] - uav0[1]
196             dist_h = float(np.hypot(dx, dy))
197             if dist_h < 1e-6:
198                 continue
199             theta = float(np.arctan2(dy, dx))
200             dz = uav0[2] - tgt[2]
201             if dz <= 2:
202                 continue
203             tf = float(np.sqrt(2.0 * dz / G))
204             tf_max = float(np.sqrt(2.0 * max(uav0[2] - 5.0, 1.0) / G) - 0.05)
205             if tf < 0.2 or tf > tf_max:
206                 continue
207             td = te - tf
208             if td < 0:

```

```

209         continue
210     speed = dist_h / te
211     if not (70.0 <= speed <= 140.0):
212         continue
213     cands.append(make_bomb(uav0, theta, speed, td, tf))
214 return cands
215
216
217 def fuse_max(uav0):
218     return float(np.sqrt(2.0 * max(uav0[2] - 5.0, 1.0) / G) - 0.05)
219
220
221 def refine_bomb(bomb, uav0, p_missile, dt=0.03):
222     best_b, best_J = bomb, union_duration([bomb], p_missile, dt=dt)
223     theta0, v0, td0, tf0 = bomb["theta"], bomb["speed"], bomb["t_drop"], bomb["t_fuse"]
224     tf_hi = fuse_max(uav0)
225     for dth in np.linspace(-0.06, 0.06, 7):
226         for dv in np.linspace(-8, 8, 5):
227             for dtd in np.linspace(-0.8, 0.8, 7):
228                 for dtf in np.linspace(-0.8, 0.8, 7):
229                     th = theta0 + dth
230                     sp = float(np.clip(v0 + dv, 70, 140))
231                     td = max(0.0, td0 + dtd)
232                     tf = float(np.clip(tf0 + dtf, 0.25, tf_hi))
233                     b = make_bomb(uav0, th, sp, td, tf)
234                     if b["det"][2] <= 1:
235                         continue
236                     J = union_duration([b], p_missile, dt=dt)
237                     if J > best_J:
238                         best_J, best_b = J, b
239     best_b = dict(best_b)
240     best_b["J"] = union_duration([best_b], p_missile, dt=0.01)
241     return best_b
242
243
244 def grid_candidates(uav0, p_missile, te_range=None):
245     """视线反推失败时的航向-速度-时序网格兜底。"""
246     base = float(np.arctan2(FAKE[1] - uav0[1], FAKE[0] - uav0[0]))
247     # 也搜索指向真目标走廊的航向
248     base2 = float(np.arctan2(200.0 - uav0[1], 0.0 - uav0[0]))
249     t_hit = missile_hit_time(p_missile)
250     td_max = 45.0 if te_range is None else max(5.0, te_range[1])
251     tf_hi = fuse_max(uav0)
252     cands = []
253     thetas = np.unique(
254         np.concatenate(
255             [

```

```

256         np.linspace(base - 1.4, base + 1.4, 11),
257         np.linspace(base2 - 0.8, base2 + 0.8, 9),
258     ]
259 )
260 )
261 for theta in thetas:
262     for speed in [70, 90, 110, 130, 140]:
263         for td in np.linspace(0.0, min(td_max, t_hit - 2), 10):
264             for tf in np.linspace(0.4, tf_hi, 12):
265                 b = make_bomb(uav0, theta, speed, td, tf)
266                 if b["det"][2] <= 1:
267                     continue
268                 cands.append(b)
269 return cands
270
271
272 def optimize_single_uav(uav_name, missile_name, n_keep=8, te_range=None):
273     uav0, p_m = UAVS[uav_name], MISSILES[missile_name]
274     t_hit = missile_hit_time(p_m)
275     if te_range is None:
276         te_grid = np.linspace(1.0, min(55.0, t_hit - 1), 60)
277     else:
278         te_grid = np.linspace(te_range[0], te_range[1], 45)
279     cands = los_candidates(uav0, p_m, te_grid=te_grid)
280     scored = []
281     for b in cands:
282         J = union_duration([b], p_m, dt=0.05)
283         if J > 0.05:
284             scored.append((J, b))
285     # 视线构造无解或过少时, 启用网格搜索兜底
286     if len(scored) < 2:
287         for b in grid_candidates(uav0, p_m, te_range):
288             J = union_duration([b], p_m, dt=0.06)
289             if J > 0.05:
290                 scored.append((J, b))
291     scored.sort(key=lambda x: -x[0])
292     picked = []
293     for J, b in scored:
294         if all(abs(b["t_det"] - p["t_det"]) > 1.2 for p in picked):
295             picked.append(b)
296         if len(picked) >= n_keep:
297             break
298     if not picked and scored:
299         picked = [scored[0][1]]
300     refined = [refine_bomb(b, uav0, p_m) for b in picked]
301     refined.sort(key=lambda b: -b["J"])
302     return refined

```

```

303
304
305 print("M1 命中时间 = {:.4f} s".format(missile_hit_time(MISSILES["M1"])))
306 print("M2 命中时间 = {:.4f} s".format(missile_hit_time(MISSILES["M2"])))
307 print("M3 命中时间 = {:.4f} s".format(missile_hit_time(MISSILES["M3"])))
308
309 # ===== Problem 1 =====
310 v1, td1, tf1 = 120.0, 1.5, 3.6
311 h1 = heading_to_fake(FY1)
312 th1 = float(np.arctan2(h1[1], h1[0]))
313 b1 = make_bomb(FY1, th1, v1, td1, tf1)
314 J1 = union_duration([b1], M1, dt=0.01)
315 pd1, pe1 = b1["drop"], b1["det"]
316
317 print("===== 问题 1 =====")
318 print("飞行速度: {:.1f} m/s".format(v1))
319 print("航向:", h1)
320 print("投放时刻: {:.1f} s, 起爆时刻: {:.1f} s".format(td1, td1 + tf1))
321 print("投放点: ({:.3f}, {:.3f}, {:.3f})".format(*pd1))
322 print("起爆点: ({:.3f}, {:.3f}, {:.3f})".format(*pe1))
323 print("有效遮蔽时长 J1 = {:.2f} s".format(J1))
324
325 ts, dists, mask, _ = shielding_series([b1], M1, dt=0.01)
326 fig, ax = plt.subplots(figsize=(8, 3.5))
327 ax.plot(ts, dists, label="球心到视线距离")
328 ax.axhline(R_SMOKE, color="r", ls="--", label="有效半径 10 m")
329 ax.fill_between(ts, 0, R_SMOKE, where=mask, color="C0", alpha=0.25, label="有效遮蔽区间")
330 ax.set_xlabel("时间 t / s"); ax.set_ylabel("距离 / m")
331 ax.set_title("问题1: 烟幕球心到视线距离随时间变化")
332 ax.legend(); ax.grid(True, alpha=0.3)
333 plt.tight_layout()
334 fig.savefig(IMG_DIR / "问题1_遮蔽距离随时间变化.png", dpi=180, bbox_inches="tight")
335 fig.savefig(FIG_DIR / "q1_distance_time.png", dpi=180, bbox_inches="tight")
336 plt.show()
337
338 # ===== Problem 2 =====
339 print("开始优化问题 2 ...")
340 cand_s_q2 = optimize_single_uav("FY1", "M1", n_keep=6, te_range=(1.0, 25.0))
341 best2 = cand_s_q2[0]
342 theta2, v2, td2, tf2 = best2["theta"], best2["speed"], best2["t_drop"], best2["t_fuse"]
343 h2 = best2["heading"]
344 pd2, pe2 = best2["drop"], best2["det"]
345 J2 = best2["J"]
346
347 print("\n===== 问题 2 =====")
348 print("速度: {:.2f} m/s, 航向角: {:.6f} rad".format(v2, theta2))
349 print("航向单位向量:", h2)

```

```

350 print("投放: {:.3f} s, 引信: {:.3f} s, 起爆: {:.3f} s".format(td2, tf2, td2 + tf2))
351 print("投放点: ({:.3f}, {:.3f}, {:.3f})".format(*pd2))
352 print("起爆点: ({:.3f}, {:.3f}, {:.3f})".format(*pe2))
353 print("有效遮蔽时长 J2 = {:.2f} s (相对问题1 + {:.2f} s)".format(J2, J2 - J1))
354
355 ts2, d2, m2, _ = shielding_series([best2], M1, dt=0.01)
356 fig, ax = plt.subplots(figsize=(8, 3.5))
357 ax.plot(ts2, d2); ax.axhline(R_SMOKE, color="r", ls="--")
358 ax.fill_between(ts2, 0, R_SMOKE, where=m2, color="C2", alpha=0.28)
359 ax.set_xlabel("时间 t / s"); ax.set_ylabel("距离 / m")
360 ax.set_title("问题2: 烟幕球心到视线距离随时间变化")
361 ax.grid(True, alpha=0.3); plt.tight_layout()
362 fig.savefig(IMG_DIR / "问题2_遮蔽距离随时间变化.png", dpi=180, bbox_inches="tight")
363 fig.savefig(FIG_DIR / "q2_distance_time.png", dpi=180, bbox_inches="tight")
364 plt.show()
365
366 # ===== Problem 3 =====
367 def optimize_q3():
368     uav0, p_m = UAVS["FY1"], MISSILES["M1"]
369     singles = optimize_single_uav("FY1", "M1", n_keep=12, te_range=(1.0, 35.0))
370     best = {"J": 0.0, "theta": None, "speed": None, "bombs": None}
371
372     # 构造基线: 最优单弹航迹上错开投放三枚
373     if singles:
374         s0 = singles[0]
375         theta, speed = s0["theta"], s0["speed"]
376         for g12 in np.linspace(1.0, 5.0, 9):
377             for g23 in np.linspace(1.0, 6.0, 9):
378                 for tf2 in np.linspace(0.5, fuse_max(uav0), 8):
379                     for tf3 in np.linspace(0.5, fuse_max(uav0), 8):
380                         drops = [s0["t_drop"], s0["t_drop"] + g12, s0["t_drop"] + g12 + g23]
381                         bombs = [
382                             make_bomb(uav0, theta, speed, drops[0], s0["t_fuse"], 1),
383                             make_bomb(uav0, theta, speed, drops[1], tf2, 2),
384                             make_bomb(uav0, theta, speed, drops[2], tf3, 3),
385                         ]
386                         if any(b["det"][2] <= 1 for b in bombs):
387                             continue
388                         tes = sorted(b["t_det"] for b in bombs)
389                         if tes[1] - tes[0] < 0.8 or tes[2] - tes[1] < 0.8:
390                             continue
391                         J = union_duration(bombs, p_m, dt=0.04)
392                         if J > best["J"]:
393                             best = {"J": J, "theta": theta, "speed": speed, "bombs": bombs}
394
395     for seed in singles[:6]:
396         theta, speed = seed["theta"], seed["speed"]

```

```

397     for td1 in np.linspace(max(0.0, seed["t_drop"] - 2), seed["t_drop"] + 2, 7):
398         for g12 in np.linspace(1.0, 6.0, 8):
399             for g23 in np.linspace(1.0, 8.0, 8):
400                 drops = [td1, td1 + g12, td1 + g12 + g23]
401                 tf_hi = fuse_max(uav0)
402                 for tf1 in np.linspace(max(0.4, seed["t_fuse"] - 1.5), min(tf_hi, seed["
t_fuse"] + 1.5), 6):
403                     for tf2 in np.linspace(0.5, tf_hi, 7):
404                         for tf3 in np.linspace(0.5, tf_hi, 7):
405                             bombs = [
406                                 make_bomb(uav0, theta, speed, drops[0], tf1, 1),
407                                 make_bomb(uav0, theta, speed, drops[1], tf2, 2),
408                                 make_bomb(uav0, theta, speed, drops[2], tf3, 3),
409                             ]
410                             if any(b["det"][2] <= 1 for b in bombs):
411                                 continue
412                             tes = sorted(b["t_det"] for b in bombs)
413                             if tes[1] - tes[0] < 0.8 or tes[2] - tes[1] < 0.8:
414                                 continue
415                             J = union_duration(bombs, p_m, dt=0.05)
416                             if J > best["J"]:
417                                 best = {"J": J, "theta": theta, "speed": speed, "bombs":
bombs}
418
419 if len(singles) >= 3:
420     for i in range(len(singles)):
421         for j in range(i + 1, len(singles)):
422             for k in range(j + 1, len(singles)):
423                 trio = sorted([singles[i], singles[j], singles[k]], key=lambda b: b["t_drop
"])
424                 if trio[1]["t_drop"] - trio[0]["t_drop"] < 1 or trio[2]["t_drop"] - trio
[1]["t_drop"] < 1:
425                     continue
426                 theta = trio[0]["theta"]
427                 speed = float(np.clip(np.mean([b["speed"] for b in trio]), 70, 140))
428                 bombs = [
429                     make_bomb(uav0, theta, speed, trio[0]["t_drop"], trio[0]["t_fuse"], 1),
430                     make_bomb(uav0, theta, speed, trio[1]["t_drop"], trio[1]["t_fuse"], 2),
431                     make_bomb(uav0, theta, speed, trio[2]["t_drop"], trio[2]["t_fuse"], 3),
432                 ]
433                 if any(b["det"][2] <= 1 for b in bombs):
434                     continue
435                 J = union_duration(bombs, p_m, dt=0.04)
436                 if J > best["J"]:
437                     best = {"J": J, "theta": theta, "speed": speed, "bombs": bombs}
438
439 theta, speed, bombs0 = best["theta"], best["speed"], best["bombs"]

```

```

440     for dth in np.linspace(-0.05, 0.05, 5):
441         for dv in np.linspace(-10, 10, 5):
442             bombs = []
443             ok = True
444             for idx, b0 in enumerate(bombs0, 1):
445                 b = make_bomb(
446                     uav0, theta + dth, float(np.clip(speed + dv, 70, 140)),
447                     b0["t_drop"], b0["t_fuse"], idx,
448                 )
449                 if b["det"][2] <= 1:
450                     ok = False
451                     break
452                 bombs.append(b)
453             if not ok:
454                 continue
455             if bombs[1]["t_drop"] - bombs[0]["t_drop"] < 1 or bombs[2]["t_drop"] - bombs[1]["
t_drop"] < 1:
456                 continue
457             J = union_duration(bombs, p_m, dt=0.03)
458             if J > best["J"]:
459                 best = {
460                     "J": J,
461                     "theta": theta + dth,
462                     "speed": float(np.clip(speed + dv, 70, 140)),
463                     "bombs": bombs,
464                 }
465
466             best["J"] = union_duration(best["bombs"], p_m, dt=0.01)
467         return best
468
469
470 print("开始优化问题 3 ...")
471 q3 = optimize_q3()
472 J3 = q3["J"]
473 print("===== 问题 3 =====")
474 print("速度: {:.2f} m/s, 航向角: {:.6f} rad".format(q3["speed"], q3["theta"]))
475 for b in q3["bombs"]:
476     print(
477         "弹{:}: td={:.3f}, tf={:.3f}, te={:.3f}, det={}".format(
478             b["id"], b["t_drop"], b["t_fuse"], b["t_det"],
479             ", ".join("{:.2f}".format(x) for x in b["det"]),
480         )
481     )
482 print("并集有效遮蔽时长 J3 = {:.2f} s".format(J3))
483
484 ts3, d3, m3, _ = shielding_series(q3["bombs"], M1, dt=0.01)
485 fig, ax = plt.subplots(figsize=(9, 3.8))

```

```

486 ax.plot(ts3, d3, label="到视线最短距离(多弹取最小)")
487 ax.axhline(R_SMOKE, color="r", ls="--", label="10 m")
488 ax.fill_between(ts3, 0, R_SMOKE, where=m3, color="C2", alpha=0.28, label="有效遮蔽")
489 for b in q3["bombs"]:
490     ax.axvline(b["t_det"], color="gray", ls=":", alpha=0.7)
491 ax.set_xlabel("时间 t / s"); ax.set_ylabel("距离 / m")
492 ax.set_title("问题3: 三弹协同遮蔽距离曲线")
493 ax.legend(); ax.grid(True, alpha=0.3); plt.tight_layout()
494 fig.savefig(IMG_DIR / "问题3_遮蔽距离随时间变化.png", dpi=180, bbox_inches="tight")
495 fig.savefig(FIG_DIR / "q3_distance_time.png", dpi=180, bbox_inches="tight")
496 plt.show()
497
498 # ===== Problem 4 =====
499 def optimize_q4():
500     p_m = MISSILES["M1"]
501     windows = [(1.0, 12.0), (8.0, 25.0), (18.0, 45.0)]
502     names = ["FY1", "FY2", "FY3"]
503     singles = []
504     for name, wr in zip(names, windows):
505         print(" 优化", name, "...")
506         cand_s = optimize_single_uav(name, "M1", n_keep=5, te_range=wr)
507         if not cand_s or cand_s[0]["J"] < 0.1:
508             cand_s = optimize_single_uav(name, "M1", n_keep=5, te_range=(1.0, 55.0))
509         singles.append(cand_s[0] if cand_s else None)
510
511     for i, name in enumerate(names):
512         if singles[i] is None or singles[i]["J"] < 0.05:
513             print(" 重试", name)
514             cand_s = optimize_single_uav(name, "M1", n_keep=8)
515             singles[i] = cand_s[0]
516
517     bombs = [dict(s) for s in singles]
518     for b in bombs:
519         b["id"] = 1
520     best_J = union_duration(bombs, p_m, dt=0.01)
521     params = np.array([b["theta"], b["speed"], b["t_drop"], b["t_fuse"]] for b in bombs)
522
523     rng = np.random.default_rng(2025)
524     for _ in range(2500):
525         trial = params.copy()
526         k = int(rng.integers(0, 3))
527         trial[k, 0] += rng.normal(0, 0.05)
528         trial[k, 1] = float(np.clip(trial[k, 1] + rng.normal(0, 5), 70, 140))
529         trial[k, 2] = max(0.0, trial[k, 2] + rng.normal(0, 1.0))
530         trial[k, 3] = float(
531             np.clip(trial[k, 3] + rng.normal(0, 0.8), 0.25, fuse_max(UAVS[names[k]]))
532         )

```

```

533     new_bombs = []
534     ok = True
535     for i, name in enumerate(names):
536         b = make_bomb(UAVS[name], trial[i, 0], trial[i, 1], trial[i, 2], trial[i, 3], 1)
537         if b["det"][2] <= 1:
538             ok = False
539             break
540         new_bombs.append(b)
541     if not ok:
542         continue
543     J = union_duration(new_bombs, p_m, dt=0.04)
544     if J > best_J:
545         best_J = union_duration(new_bombs, p_m, dt=0.01)
546         params = trial
547         bombs = new_bombs
548
549     strategies = []
550     for i, name in enumerate(names):
551         b = bombs[i]
552         strategies.append({
553             "uav": name,
554             "theta": b["theta"],
555             "speed": b["speed"],
556             "bomb": b,
557             "Ji": union_duration([b], p_m, dt=0.01),
558         })
559     return {"strategies": strategies, "bombs": bombs, "J": best_J}
560
561
562 print("开始优化问题 4 ...")
563 q4 = optimize_q4()
564 J4 = q4["J"]
565 print("===== 问题 4 =====")
566 for s in q4["strategies"]:
567     b = s["bomb"]
568     print(
569         "{: v={:.2f}, theta={:.4f}, td={:.3f}, te={:.3f}, Ji={:.2f}".format(
570             s["uav"], s["speed"], s["theta"], b["t_drop"], b["t_det"], s["Ji"]
571         )
572     )
573 print("并集有效遮蔽时长 J4 = {:.2f} s".format(J4))
574
575 ts4, d4, m4, _ = shielding_series(q4["bombs"], M1, dt=0.01)
576 fig, ax = plt.subplots(figsize=(9, 3.8))
577 ax.plot(ts4, d4); ax.axhline(R_SMOKE, color="r", ls="--")
578 ax.fill_between(ts4, 0, R_SMOKE, where=m4, color="C2", alpha=0.28)
579 for s in q4["strategies"]:

```

```

580     ax.axvline(s["bomb"]["t_det"], ls=":", alpha=0.8, label=s["uav"] + " 起爆")
581 ax.set_xlabel("时间 t / s"); ax.set_ylabel("距离 / m")
582 ax.set_title("问题4: 三机协同遮蔽距离曲线")
583 ax.legend(fontsize=8); ax.grid(True, alpha=0.3); plt.tight_layout()
584 fig.savefig(IMG_DIR / "问题4_遮蔽距离随时间变化.png", dpi=180, bbox_inches="tight")
585 fig.savefig(FIG_DIR / "q4_distance_time.png", dpi=180, bbox_inches="tight")
586 plt.show()
587
588 # ===== Problem 5 =====
589 def optimize_multi_on_uav(uav_name, missile_name, n_bombs):
590     uav0, p_m = UAVS[uav_name], MISSILES[missile_name]
591     singles = optimize_single_uav(uav_name, missile_name, n_keep=10)
592     if not singles:
593         return {"uav": uav_name, "missile": missile_name, "bombs": [], "J": 0.0}
594
595     best = {"J": 0.0, "bombs": None, "theta": None, "speed": None}
596     pool = singles[:8]
597     for comb in combinations(range(len(pool)), min(n_bombs, len(pool))):
598         chosen = sorted([pool[i] for i in comb], key=lambda b: b["t_drop"])
599         ok_gap = all(
600             chosen[i + 1]["t_drop"] - chosen[i]["t_drop"] >= 1.0
601             for i in range(len(chosen) - 1)
602         )
603         if not ok_gap:
604             continue
605         theta = chosen[0]["theta"]
606         speed = float(np.clip(np.mean([b["speed"] for b in chosen]), 70, 140))
607         bombs = [
608             make_bomb(uav0, theta, speed, b["t_drop"], b["t_fuse"], i + 1)
609             for i, b in enumerate(chosen)
610         ]
611         if any(b["det"][2] <= 1 for b in bombs):
612             continue
613         J = union_duration(bombs, p_m, dt=0.05)
614         if J > best["J"]:
615             best = {"J": J, "bombs": bombs, "theta": theta, "speed": speed}
616
617     if best["bombs"] is None:
618         b0 = singles[0]
619         bombs = []
620         td = b0["t_drop"]
621         for i in range(n_bombs):
622             bombs.append(make_bomb(uav0, b0["theta"], b0["speed"], td, b0["t_fuse"], i + 1))
623             td += max(DROP_GAP, 2.5)
624         best = {
625             "J": union_duration(bombs, p_m, dt=0.01),
626             "bombs": bombs,

```

```

627         "theta": b0["theta"],
628         "speed": b0["speed"],
629     }
630
631     theta, speed, bombs0 = best["theta"], best["speed"], best["bombs"]
632     for dth in np.linspace(-0.05, 0.05, 5):
633         for dv in [-8, 0, 8]:
634             bombs = [
635                 make_bomb(
636                     uav0, theta + dth, float(np.clip(speed + dv, 70, 140)),
637                     b["t_drop"], b["t_fuse"], i + 1,
638                 )
639                 for i, b in enumerate(bombs0)
640             ]
641             if any(b["det"][2] <= 1 for b in bombs):
642                 continue
643             if any(bombs[i + 1]["t_drop"] - bombs[i]["t_drop"] < 1 for i in range(len(bombs) -
644 1)):
645                 continue
646             J = union_duration(bombs, p_m, dt=0.03)
647             if J > best["J"]:
648                 best = {
649                     "J": J,
650                     "bombs": bombs,
651                     "theta": theta + dth,
652                     "speed": float(np.clip(speed + dv, 70, 140)),
653                 }
654             best["J"] = union_duration(best["bombs"], p_m, dt=0.01)
655             best["uav"] = uav_name
656             best["missile"] = missile_name
657             return best
658
659 def optimize_q5():
660     # FY1/FY4 靠近 M1 来向; FY2 偏北配合 M2; FY3/FY5 偏南配合 M3
661     assign = {"FY1": "M1", "FY4": "M1", "FY2": "M2", "FY3": "M3", "FY5": "M3"}
662     n_map = {"FY1": 2, "FY4": 2, "FY2": 3, "FY3": 2, "FY5": 2}
663     strategies = []
664     by_m = {"M1": [], "M2": [], "M3": []}
665     for uav, missile in assign.items():
666         print(" {} -> {}, n={} ...".format(uav, missile, n_map[uav]))
667         s = optimize_multi_on_uav(uav, missile, n_map[uav])
668         strategies.append(s)
669         by_m[missile].extend(s["bombs"])
670     durations = {m: union_duration(bs, MISSILES[m], dt=0.01) for m, bs in by_m.items()}
671     return {
672         "assign": assign,

```

```

673     "n_map": n_map,
674     "strategies": strategies,
675     "by_missile": by_m,
676     "durations": durations,
677     "total": float(sum(durations.values())),
678 }
679
680
681 print("开始优化问题 5 ...")
682 q5 = optimize_q5()
683 print("===== 问题 5 =====")
684 for m, J in q5["durations"].items():
685     print("{} 有效遮蔽时长 = {:.2f} s (弹数 {}) ".format(m, J, len(q5["by_missile"][m])))
686 print("三导弹合计 J5 = {:.2f} s".format(q5["total"]))
687
688 fig, axes = plt.subplots(3, 1, figsize=(9, 9), sharex=False)
689 for ax, m in zip(axes, ["M1", "M2", "M3"]):
690     ts, d, mask, J = shielding_series(q5["by_missile"][m], MISSILES[m], dt=0.02)
691     if len(ts):
692         ax.plot(ts, d)
693         ax.axhline(R_SMOKE, color="r", ls="--")
694         ax.fill_between(ts, 0, R_SMOKE, where=mask, color="C2", alpha=0.28)
695         ax.set_title("{} 遮蔽距离曲线 (J={:.2f}s) ".format(m, J))
696         ax.set_ylabel("距离 / m")
697         ax.grid(True, alpha=0.3)
698 axes[-1].set_xlabel("时间 t / s")
699 plt.tight_layout()
700 fig.savefig(IMG_DIR / "问题5_三导弹遮蔽距离曲线.png", dpi=180, bbox_inches="tight")
701 fig.savefig(FIG_DIR / "q5_distance_time.png", dpi=180, bbox_inches="tight")
702 plt.show()
703
704 # ===== Export Excel and summary figures =====
705 def ang_deg(heading):
706     return float(np.degrees(np.arctan2(heading[1], heading[0])))
707
708
709 def rows_from_bombs(uav, bombs, duration, missile=None):
710     rows = []
711     for b in bombs:
712         row = {
713             "无人机编号": uav,
714             "无人机运动方向": round(ang_deg(b["heading"]), 3),
715             "无人机运动速度 (m/s)": round(b["speed"], 3),
716             "烟幕干扰弹编号": b["id"],
717             "烟幕干扰弹投放点的x坐标 (m)": round(float(b["drop"][0]), 3),
718             "烟幕干扰弹投放点的y坐标 (m)": round(float(b["drop"][1]), 3),
719             "烟幕干扰弹投放点的z坐标 (m)": round(float(b["drop"][2]), 3),

```

```

720         "烟雾干扰弹起爆点的x坐标 (m)": round(float(b["det"][0]), 3),
721         "烟雾干扰弹起爆点的y坐标 (m)": round(float(b["det"][1]), 3),
722         "烟雾干扰弹起爆点的z坐标 (m)": round(float(b["det"][2]), 3),
723         "有效干扰时长 (s)": round(float(duration), 3),
724     }
725     if missile is not None:
726         row["干扰的导弹编号"] = missile
727     rows.append(row)
728     return rows
729
730
731 df1 = pd.DataFrame(rows_from_bombs("FY1", q3["bombs"], J3))
732 df1 = df1.drop(columns=["无人机编号"])
733 df1.to_excel("result1.xlsx", index=False)
734 df1.to_excel(TAB_DIR / "问题3_result1.xlsx", index=False)
735
736 rows2 = []
737 for s in q4["strategies"]:
738     rows2 += rows_from_bombs(s["uav"], [s["bomb"]], J4)
739 df2 = pd.DataFrame(rows2)
740 df2.to_excel("result2.xlsx", index=False)
741 df2.to_excel(TAB_DIR / "问题4_result2.xlsx", index=False)
742
743 rows3 = []
744 for s in q5["strategies"]:
745     rows3 += rows_from_bombs(
746         s["uav"], s["bombs"], q5["durations"][s["missile"]], s["missile"]
747     )
748 df3 = pd.DataFrame(rows3)
749 df3.to_excel("result3.xlsx", index=False)
750 df3.to_excel(TAB_DIR / "问题5_result3.xlsx", index=False)
751
752 print("已保存 result1.xlsx / result2.xlsx / result3.xlsx")
753 print(df1)
754 print(df2)
755 print(df3)
756
757 fig, ax = plt.subplots(figsize=(8, 4.2))
758 names = ["问题1", "问题2", "问题3", "问题4", "问题5合计"]
759 vals = [J1, J2, J3, J4, q5["total"]]
760 colors = ["#4C78A8", "#F58518", "#54A24B", "#E45756", "#B279A2"]
761 bars = ax.bar(names, vals, color=colors, width=0.6)
762 for b, v in zip(bars, vals):
763     ax.text(b.get_x() + b.get_width() / 2, v + 0.08, "{:.2f}s".format(v), ha="center", va="
764         bottom")
765 ax.set_ylabel("有效遮蔽时长 / s")
766 ax.set_title("问题1-5 有效遮蔽时长对比")

```

```

766 ax.set_ylim(0, max(vals) * 1.2)
767 ax.grid(True, axis="y", alpha=0.3)
768 plt.tight_layout()
769 fig.savefig(IMG_DIR / "问题1至5_遮蔽时长对比图.png", dpi=180, bbox_inches="tight")
770 fig.savefig(FIG_DIR / "q1_to_q5_duration_compare.png", dpi=180, bbox_inches="tight")
771 plt.show()
772
773 fig, ax = plt.subplots(figsize=(8, 4.5))
774 t_mis = np.linspace(0, missile_hit_time(M1), 100)
775 mis = missile_pos(t_mis, M1)
776 ax.plot(mis[:, 0], mis[:, 1], "r-", lw=1.2, label="M1 水平投影")
777 ax.scatter(0, 0, c="k", marker="x", s=50, label="假目标")
778 ax.add_patch(plt.Circle((0, 200), 7, fill=False, color="g", label="真目标"))
779 for b in q3["bombs"]:
780     ax.scatter(b["det"][0], b["det"][1], marker="*", s=90, label="弹{}起爆".format(b["id"]))
781     ax.add_patch(plt.Circle((b["det"][0], b["det"][1]), 10, alpha=0.12))
782 ax.set_aspect("equal", adjustable="datalim")
783 ax.set_xlabel("X / m"); ax.set_ylabel("Y / m")
784 ax.set_title("问题3: 三弹起爆点俯视图")
785 ax.legend(fontsize=8); ax.grid(True, alpha=0.3)
786 plt.tight_layout()
787 fig.savefig(IMG_DIR / "问题3_起爆点俯视图.png", dpi=180, bbox_inches="tight")
788 fig.savefig(FIG_DIR / "q3_topview.png", dpi=180, bbox_inches="tight")
789 plt.show()
790
791 fig, ax = plt.subplots(figsize=(8, 4.5))
792 ax.plot(mis[:, 0], mis[:, 1], "r-", lw=1.2, label="M1")
793 ax.scatter(0, 0, c="k", marker="x", s=50)
794 ax.add_patch(plt.Circle((0, 200), 7, fill=False, color="g"))
795 for s in q4["strategies"]:
796     b = s["bomb"]
797     ax.scatter(UAVS[s["uav"]][0], UAVS[s["uav"]][1], marker="^", s=60, label=s["uav"] + "起点")
798     ax.scatter(b["det"][0], b["det"][1], marker="*", s=90, label=s["uav"] + "起爆")
799 ax.set_aspect("equal", adjustable="datalim")
800 ax.set_xlabel("X / m"); ax.set_ylabel("Y / m")
801 ax.set_title("问题4: 三机起爆点俯视图")
802 ax.legend(fontsize=7, ncol=2); ax.grid(True, alpha=0.3)
803 plt.tight_layout()
804 fig.savefig(IMG_DIR / "问题4_起爆点俯视图.png", dpi=180, bbox_inches="tight")
805 fig.savefig(FIG_DIR / "q4_topview.png", dpi=180, bbox_inches="tight")
806 plt.show()
807
808 summary = pd.DataFrame([
809     {"问题": "问题1", "场景": "FY1×1 vs M1(给定)", "有效遮蔽时长(s)": round(J1, 2)},
810     {"问题": "问题2", "场景": "FY1×1 vs M1(优化)", "有效遮蔽时长(s)": round(J2, 2)},
811     {"问题": "问题3", "场景": "FY1×3 vs M1", "有效遮蔽时长(s)": round(J3, 2)},
812     {"问题": "问题4", "场景": "FY1/2/3×1 vs M1", "有效遮蔽时长(s)": round(J4, 2)},

```

```

813     {"问题": "问题5-M1", "场景": "分配机群 vs M1", "有效遮蔽时长(s)": round(q5["durations"]["M1"
814         ], 2)},
815     {"问题": "问题5-M2", "场景": "分配机群 vs M2", "有效遮蔽时长(s)": round(q5["durations"]["M2"
816         ], 2)},
817     {"问题": "问题5-M3", "场景": "分配机群 vs M3", "有效遮蔽时长(s)": round(q5["durations"]["M3"
818         ], 2)},
819     {"问题": "问题5合计", "场景": "五机 vs 三导弹", "有效遮蔽时长(s)": round(q5["total"], 2)},
820 ]})
821 summary.to_excel(TAB_DIR / "问题1至5_结果汇总表.xlsx", index=False)
822 print(summary.to_string(index=False))
823 print("完成。图片目录:", IMG_DIR.resolve())

```

Listing 1: 问题 1–5 完整求解程序 (solve_all_problems.py)

附录 C 问题 3–5 补充作图程序

下列程序根据 result1.xlsx–result3.xlsx 生成论文中问题 3–5 的补充图表:

python code/gen_q345_figures.py

```

1  # -*- coding: utf-8 -*-
2  """为问题3–5生成补充图表（与论文详细求解配套）。"""
3  from __future__ import annotations
4
5  import json
6  from pathlib import Path
7
8  import matplotlib.pyplot as plt
9  import numpy as np
10 import pandas as pd
11
12 plt.rcParams["font.sans-serif"] = ["SimHei", "Microsoft YaHei", "DejaVu Sans"]
13 plt.rcParams["axes.unicode_minus"] = False
14
15 ROOT = Path(__file__).resolve().parents[1]
16 FIG = ROOT / "figures"
17 IMG = ROOT / "图片"
18 TAB = ROOT / "表格"
19 for d in (FIG, IMG, TAB):
20     d.mkdir(exist_ok=True)
21
22 # 从 notebook 逻辑复用: 直接读 result 并重建关键量
23 G, V_MISSILE, V_SINK, R_SMOKE, T_SMOKE = 9.8, 300.0, 3.0, 10.0, 20.0
24 FAKE = np.array([0.0, 0.0, 0.0])
25 TRUE_CENTER = np.array([0.0, 200.0, 5.0])
26 MISSILES = {

```

```

27     "M1": np.array([20000.0, 0.0, 2000.0]),
28     "M2": np.array([19000.0, 600.0, 2100.0]),
29     "M3": np.array([18000.0, -600.0, 1900.0]),
30 }
31 UAVS = {
32     "FY1": np.array([17800.0, 0.0, 1800.0]),
33     "FY2": np.array([12000.0, 1400.0, 1400.0]),
34     "FY3": np.array([6000.0, -3000.0, 700.0]),
35     "FY4": np.array([11000.0, 2000.0, 1800.0]),
36     "FY5": np.array([13000.0, -2000.0, 1300.0]),
37 }
38
39
40 def unit(v):
41     n = float(np.linalg.norm(v))
42     return v / n if n > 1e-12 else v
43
44
45 def missile_hit_time(p0):
46     return float(np.linalg.norm(p0 - FAKE) / V_MISSILE)
47
48
49 def missile_pos(t, p0):
50     e = unit(FAKE - p0)
51     if np.isscalar(t):
52         return p0 + V_MISSILE * float(t) * e
53     t = np.asarray(t, dtype=float)
54     return p0[None, :] + (V_MISSILE * t[:, None] * e[None, :])
55
56
57 def point_to_segment_dist_batch(p, a, b):
58     ab = b[None, :] - a
59     L2 = np.sum(ab * ab, axis=1)
60     s = np.zeros(len(p))
61     ok = L2 > 1e-18
62     ap = p - a
63     s[ok] = np.clip(np.sum(ap[ok] * ab[ok], axis=1) / L2[ok], 0.0, 1.0)
64     return np.linalg.norm(p - (a + s[:, None] * ab), axis=1)
65
66
67 def heading_from_deg(deg):
68     th = np.deg2rad(deg)
69     return np.array([np.cos(th), np.sin(th), 0.0])
70
71
72 def bombs_from_result_row(row, uav_col=None):
73     uav = row[uav_col] if uav_col else "FY1"

```

```

74     uav0 = UAVS[uav]
75     h = heading_from_deg(row["无人机运动方向"])
76     speed = float(row["无人机运动速度 (m/s)"])
77     pd = np.array(
78         [
79             row["烟幕干扰弹投放点的x坐标 (m)"],
80             row["烟幕干扰弹投放点的y坐标 (m)"],
81             row["烟幕干扰弹投放点的z坐标 (m)"],
82         ]
83     )
84     pe = np.array(
85         [
86             row["烟幕干扰弹起爆点的x坐标 (m)"],
87             row["烟幕干扰弹起爆点的y坐标 (m)"],
88             row["烟幕干扰弹起爆点的z坐标 (m)"],
89         ]
90     )
91     # 反推 td, tf
92     # pd = uav0 + speed*td*h => td from horizontal
93     disp = pd - uav0
94     td = float(np.dot(disp[:2], h[:2]) / max(speed, 1e-9))
95     td = max(0.0, td)
96     # pe_xy = pd_xy + speed*tf*h_xy
97     disp2 = pe[:2] - pd[:2]
98     tf = float(np.dot(disp2, h[:2]) / max(speed, 1e-9))
99     tf = max(0.05, tf)
100    return {
101        "id": int(row["烟幕干扰弹编号"]),
102        "uav": uav,
103        "speed": speed,
104        "heading": h,
105        "t_drop": td,
106        "t_fuse": tf,
107        "t_det": td + tf,
108        "drop": pd,
109        "det": pe,
110        "missile": row.get("干扰的导弹编号", "M1"),
111    }
112
113
114    def series_multi(bombs, p_missile, dt=0.01):
115        t_hit = missile_hit_time(p_missile)
116        t0 = min(b["t_det"] for b in bombs)
117        t1 = min(max(b["t_det"] + T_SMOKE for b in bombs), t_hit)
118        ts = np.arange(t0, t1 + 1e-12, dt)
119        mpos = missile_pos(ts, p_missile)
120        min_d = np.full(len(ts), np.inf)

```

```

121 any_mask = np.zeros(len(ts), dtype=bool)
122 per = []
123 for b in bombs:
124     te, pe = b["t_det"], b["det"]
125     mask = (ts >= te) & (ts <= te + T_SMOKE)
126     d_full = np.full(len(ts), np.nan)
127     if np.any(mask):
128         idx = np.where(mask)[0]
129         c = np.column_stack(
130             [
131                 np.full(len(idx), pe[0]),
132                 np.full(len(idx), pe[1]),
133                 pe[2] - V_SINK * (ts[idx] - te),
134             ]
135         )
136         d = point_to_segment_dist_batch(c, mpos[idx], TRUE_CENTER)
137         d_full[idx] = d
138         min_d[idx] = np.minimum(min_d[idx], d)
139         any_mask[idx] |= d <= R_SMOKE
140     per.append((b, d_full, (~np.isnan(d_full)) & (d_full <= R_SMOKE)))
141 return ts, min_d, any_mask, per, float(any_mask.sum() * dt)
142
143
144 def savefig(fig, name_en, name_zh):
145     fig.savefig(FIG / name_en, dpi=180, bbox_inches="tight")
146     fig.savefig(IMG / name_zh, dpi=180, bbox_inches="tight")
147     plt.close(fig)
148
149
150 def plot_q3(df1):
151     bombs = [bombs_from_result_row(r) for _, r in df1.iterrows()]
152     for b in bombs:
153         b["missile"] = "M1"
154     ts, min_d, mask, per, J = series_multi(bombs, MISSILES["M1"], 0.01)
155
156     # 分弹距离曲线
157     fig, ax = plt.subplots(figsize=(9.2, 4.2))
158     colors = ["#4C78A8", "#F58518", "#54A24B"]
159     for (b, d_full, m), c in zip(per, colors):
160         ax.plot(ts, d_full, color=c, lw=1.4, label=f"弹{b['id']} 距离")
161         ax.axvline(b["t_det"], color=c, ls=":", alpha=0.7)
162     ax.plot(ts, min_d, "k-", lw=1.8, label="多弹取最小距离")
163     ax.axhline(10, color="crimson", ls="--", label="阈值 10 m")
164     ax.fill_between(ts, 0, 10, where=mask, color="C2", alpha=0.2, label="交集有效区")
165     ax.set_xlabel("时间 t / s")
166     ax.set_ylabel("距离 / m")
167     ax.set_title("问题3: 各弹到视线距离与交集有效区间")

```

```

168 ax.set_ylim(0, 80)
169 ax.legend(fontsize=8, ncol=2)
170 ax.grid(True, alpha=0.3)
171 fig.tight_layout()
172 savefig(fig, "q3_per_bomb_distance.png", "问题3_各弹距离与并集.png")
173
174 # 甘特式时序
175 fig, ax = plt.subplots(figsize=(9, 3.2))
176 for i, (b, _, m) in enumerate(per):
177     # 有效段
178     if m.any():
179         # 找连续段
180         idx = np.where(m)[0]
181         starts = [idx[0]]
182         for a, bb in zip(idx[:-1], idx[1:]):
183             if bb != a + 1:
184                 starts.append(bb)
185         # simpler: plot scatter of effective
186         ax.plot(ts[m], np.full(m.sum(), i + 1), "|", color=colors[i], ms=10)
187         ax.scatter([b["t_det"]], [i + 1], marker="*", s=120, color=colors[i], zorder=5)
188         ax.text(b["t_det"], i + 1.15, f"起爆{b['t_det']:.2f}s", color=colors[i], fontsize=8)
189 ax.set_yticks([1, 2, 3])
190 ax.set_yticklabels(["弹1", "弹2", "弹3"])
191 ax.set_xlabel("时间 t / s")
192 ax.set_title(f"问题3: 三弹有效遮蔽时序 (并集 J={J:.2f}s)")
193 ax.grid(True, axis="x", alpha=0.3)
194 fig.tight_layout()
195 savefig(fig, "q3_timeline.png", "问题3_有效遮蔽时序图.png")
196
197 # 与问题1/2对比
198 fig, ax = plt.subplots(figsize=(6.8, 4.0))
199 names = ["问题1\n单弹给定", "问题2\n单弹优化", "问题3\n同机三弹"]
200 vals = [1.43, 4.71, J]
201 bars = ax.bar(names, vals, color=["#4C78A8", "#F58518", "#54A24B"], width=0.55)
202 for b, v in zip(bars, vals):
203     ax.text(b.get_x() + b.get_width() / 2, v + 0.08, f"{v:.2f}s", ha="center")
204 ax.set_ylabel("有效遮蔽时长 / s")
205 ax.set_title("问题1 - 3 遮蔽时长对比")
206 ax.set_ylim(0, max(vals) * 1.25)
207 ax.grid(True, axis="y", alpha=0.3)
208 fig.tight_layout()
209 savefig(fig, "q1q2q3_duration_compare.png", "问题1至3_遮蔽时长对比图.png")
210
211 # 导出明细
212 rows = []
213 for b in bombs:
214     rows.append(

```

```

215         {
216             "弹号": b["id"],
217             "投放时刻(s)": round(b["t_drop"], 3),
218             "引信延时(s)": round(b["t_fuse"], 3),
219             "起爆时刻(s)": round(b["t_det"], 3),
220             "投放点": f"({b['drop'][0]:.2f},{b['drop'][1]:.2f},{b['drop'][2]:.2f})",
221             "起爆点": f"({b['det'][0]:.2f},{b['det'][1]:.2f},{b['det'][2]:.2f})",
222         }
223     )
224     pd.DataFrame(rows).to_excel(TAB / "问题3_三弹时序明细表.xlsx", index=False)
225     return bombs, J
226
227
228 def plot_q4(df2):
229     bombs = [bombs_from_result_row(r, "无人机编号") for _, r in df2.iterrows()]
230     ts, min_d, mask, per, J = series_multi(bombs, MISSILES["M1"], 0.01)
231     Ji = []
232     for b, _, m in per:
233         Ji.append(float(m.sum() * 0.01))
234
235     fig, ax = plt.subplots(figsize=(9.2, 4.2))
236     colors = ["#4C78A8", "#F58518", "#54A24B"]
237     for (b, d_full, m), c in zip(per, colors):
238         ax.plot(ts, d_full, color=c, lw=1.3, label=f"{b['uav']} 距离")
239         ax.axvline(b["t_det"], color=c, ls=":", alpha=0.75)
240     ax.plot(ts, min_d, "k-", lw=1.7, label="三机取最小")
241     ax.axhline(10, color="crimson", ls="--")
242     ax.fill_between(ts, 0, 10, where=mask, color="C2", alpha=0.22, label="并集有效区")
243     ax.set_ylim(0, 100)
244     ax.set_xlabel("时间 t / s")
245     ax.set_ylabel("距离 / m")
246     ax.set_title("问题4: 三机各自距离曲线与并集有效区")
247     ax.legend(fontsize=8, ncol=2)
248     ax.grid(True, alpha=0.3)
249     fig.tight_layout()
250     savefig(fig, "q4_per_uav_distance.png", "问题4_各机距离与并集.png")
251
252     # 单弹 vs 并集
253     fig, ax = plt.subplots(figsize=(7.2, 4.0))
254     names = [b["uav"] + "单弹" for b in bombs] + ["三机并集"]
255     vals = Ji + [J]
256     colors = ["#4C78A8", "#F58518", "#54A24B", "#E45756"]
257     bars = ax.bar(names, vals, color=colors, width=0.55)
258     for b, v in zip(bars, vals):
259         ax.text(b.get_x() + b.get_width() / 2, v + 0.15, f"{v:.2f}s", ha="center")
260     ax.set_ylabel("有效遮蔽时长 / s")
261     ax.set_title("问题4: 单机贡献与并集时长")

```

```

262 ax.set_ylim(0, max(vals) * 1.2)
263 ax.grid(True, axis="y", alpha=0.3)
264 fig.tight_layout()
265 savefig(fig, "q4_union_vs_single.png", "问题4_单机与并集对比图.png")
266
267 # 时序带
268 fig, ax = plt.subplots(figsize=(9, 3.0))
269 for i, (b, _, m) in enumerate(per):
270     ax.plot(ts[m], np.full(m.sum(), i + 1), "|", color=colors[i], ms=9)
271     ax.scatter([b["t_det"]], [i + 1], marker="*", s=110, color=colors[i])
272 ax.set_yticks([1, 2, 3])
273 ax.set_yticklabels([b["uav"] for b in bombs])
274 ax.set_xlabel("时间 t / s")
275 ax.set_title(f"问题4: 三机有效遮蔽时序 (并集 J={J:.2f}s)")
276 ax.grid(True, axis="x", alpha=0.3)
277 fig.tight_layout()
278 savefig(fig, "q4_timeline.png", "问题4_有效遮蔽时序图.png")
279
280 rows = []
281 for b, j in zip(bombs, Ji):
282     rows.append(
283         {
284             "无人机": b["uav"],
285             "速度(m/s)": round(b["speed"], 3),
286             "投放时刻(s)": round(b["t_drop"], 3),
287             "引信延时(s)": round(b["t_fuse"], 3),
288             "起爆时刻(s)": round(b["t_det"], 3),
289             "单弹遮蔽(s)": round(j, 3),
290             "投放点": f"({b['drop'][0]:.2f},{b['drop'][1]:.2f},{b['drop'][2]:.2f})",
291             "起爆点": f"({b['det'][0]:.2f},{b['det'][1]:.2f},{b['det'][2]:.2f})",
292         }
293     )
294 pd.DataFrame(rows).to_excel(TAB / "问题4_三机策略明细表.xlsx", index=False)
295 return bombs, Ji, J
296
297
298 def plot_q5(df3):
299     by_m = {"M1": [], "M2": [], "M3": []}
300     for _, r in df3.iterrows():
301         b = bombs_from_result_row(r, "无人机编号")
302         by_m[b["missile"]].append(b)
303
304     durations = {}
305     for m, bombs in by_m.items():
306         ts, min_d, mask, per, J = series_multi(bombs, MISSILES[m], 0.02)
307         durations[m] = J
308     fig, ax = plt.subplots(figsize=(9, 3.6))

```

```

309     ax.plot(ts, min_d, lw=1.6)
310     ax.axhline(10, color="r", ls="--")
311     ax.fill_between(ts, 0, 10, where=mask, color="C2", alpha=0.25)
312     for b in bombs:
313         ax.axvline(b["t_det"], ls=":", alpha=0.6)
314     ax.set_title(f"问题5: {m} 遮蔽距离曲线 (J={J:.2f}s, 弹数{len(bombs)}) ")
315     ax.set_xlabel("时间 t / s")
316     ax.set_ylabel("距离 / m")
317     ax.set_ylim(0, 80)
318     ax.grid(True, alpha=0.3)
319     fig.tight_layout()
320     savefig(fig, f"q5_{m.lower()}_distance.png", f"问题5_{m}_遮蔽距离曲线.png")
321
322 # 分配示意柱状
323 fig, ax = plt.subplots(figsize=(7.5, 4.2))
324 names = ["M1\n(FY1+FY4)", "M2\n(FY2)", "M3\n(FY3+FY5)", "合计"]
325 vals = [durations["M1"], durations["M2"], durations["M3"], sum(durations.values())]
326 bars = ax.bar(names, vals, color=["#4C78A8", "#F58518", "#54A24B", "#B279A2"], width=0.55)
327 for b, v in zip(bars, vals):
328     ax.text(b.get_x() + b.get_width() / 2, v + 0.2, f"{v:.2f}s", ha="center")
329 ax.set_ylabel("有效遮蔽时长 / s")
330 ax.set_title("问题5: 任务分配与各导弹遮蔽时长")
331 ax.set_ylim(0, max(vals) * 1.2)
332 ax.grid(True, axis="y", alpha=0.3)
333 fig.tight_layout()
334 savefig(fig, "q5_assignment_bar.png", "问题5_任务分配与遮蔽时长.png")
335
336 # 俯视: 五机与三导弹
337 fig, ax = plt.subplots(figsize=(9, 5.5))
338 for name, p0 in MISSILES.items():
339     t_hit = missile_hit_time(p0)
340     t = np.linspace(0, t_hit, 80)
341     traj = missile_pos(t, p0)
342     ax.plot(traj[:, 0], traj[:, 1], lw=1.2, label=f"{name}轨迹")
343     ax.scatter(p0[0], p0[1], marker="x", s=50)
344 ax.scatter(0, 0, c="k", marker="s", s=40, label="假目标")
345 ax.add_patch(plt.Circle((0, 200), 7, fill=False, color="g", lw=1.5, label="真目标"))
346 cmap = {"FY1": "C0", "FY2": "C1", "FY3": "C2", "FY4": "C3", "FY5": "C4"}
347 for _, r in df3.iterrows():
348     u = r["无人机编号"]
349     ax.scatter(UAVS[u][0], UAVS[u][1], marker="^", c=cmap[u], s=55)
350     ax.scatter(
351         r["烟幕干扰弹起爆点的x坐标 (m)",
352         r["烟幕干扰弹起爆点的y坐标 (m)",
353         marker="*",
354         c=cmap[u],
355         s=70,

```

```

356     )
357     for u, c in cmap.items():
358         ax.scatter([], [], marker="^", c=c, label=u)
359     ax.set_aspect("equal", adjustable="datalim")
360     ax.set_xlabel("X / m")
361     ax.set_ylabel("Y / m")
362     ax.set_title("问题5: 五机起点、起爆点与三导弹水平态势")
363     ax.legend(fontsize=7, ncol=3, loc="best")
364     ax.grid(True, alpha=0.3)
365     fig.tight_layout()
366     savefig(fig, "q5_topview.png", "问题5_五机三导弹俯视图.png")
367
368     pd.DataFrame(
369         [
370             {"导弹": m, "弹数": len(by_m[m]), "有效遮蔽时长(s)": round(durations[m], 2)}
371             for m in ["M1", "M2", "M3"]
372         ]
373         + [{"导弹": "合计", "弹数": len(df3), "有效遮蔽时长(s)": round(sum(durations.values()),
374             2)}]
375     ).to_excel(TAB / "问题5_各导弹结果汇总表.xlsx", index=False)
376     return durations
377
378 def main():
379     df1 = pd.read_excel(ROOT / "result1.xlsx")
380     df2 = pd.read_excel(ROOT / "result2.xlsx")
381     df3 = pd.read_excel(ROOT / "result3.xlsx")
382     print("plot q3...")
383     bombs3, J3 = plot_q3(df1)
384     print("J3", J3)
385     for b in bombs3:
386         print(b["id"], b["t_drop"], b["t_fuse"], b["t_det"])
387     print("plot q4...")
388     bombs4, Ji, J4 = plot_q4(df2)
389     print("J4", J4, Ji)
390     for b in bombs4:
391         print(b["uav"], b["t_drop"], b["t_fuse"], b["t_det"], b["speed"])
392     print("plot q5...")
393     d5 = plot_q5(df3)
394     print(d5)
395     print("done")
396
397
398 if __name__ == "__main__":
399     main()

```

Listing 2: 问题 3–5 补充作图程序 (gen_q345_figures.py)